

A PORTFOLIO BOOK

Programming in Python

A hands-on companion, from the shell to object-oriented design.

IIT MADRAS ONLINE DEGREE · COMPILED ON PORTFOLIO-V2

0

Warm-up

Lesson 0 — Getting Started with Python

What is Python?

A programming language is, in essence, a language you use to tell your computer what to do. Python is a general-purpose language that is useful for data science, automation, machine learning, and software and web development.

Consider a program that sums the first 100 numbers:

PYTHON

```
sum = 0
for number in range(1, 101):
    sum += number
print(sum)
```

Don't worry about how this works yet. Contrast using a calculator to add the numbers 1 through 100 with simply running this code — the program is *blazingly fast*. Coding lets you build solutions and applications; even the calculators used by non-coders are themselves built with code. The example programs in these lessons live in `main.py` files, which you can view via a "Show files" button.

Why learn Python?

The main reasons are **utility** and **popularity**. Python powers applications at companies like Google, Netflix, Dropbox, and Quora. A 2022 StackOverflow survey rated Python "the second most wanted language," with around 66% of roughly 65,000 respondents developing with it and wanting to continue.

Python also lets you create beautiful things like animations — coders are like musicians and painters using their tools. Python is human-readable, easy, and powerful, which makes it an ideal first language.

HOW EASY IS PYTHON?

Python's vast collection of libraries eases complex tasks. This is illustrated by the xkcd comic that appears when you enter `import antigravity` in IDLE.

Lessons

Organization

Lessons are numbered `<chapter>.<lesson>`, with about four lessons per chapter, and are best read in sequence from chapter 1 onward. Readers who are already familiar can jump ahead via the Table of Contents. Each chapter introduces one main concept (its title), though other topics also appear — for example, chapter 2 covers conditionals plus built-in functions and standard libraries. The book outline is:

- Chapter-1: Introduction to Python
- Chapter-2: Conditionals
- Chapter-3: Loops
- Chapter-4: Functions
- Chapter-5: Lists and Tuples
- Chapter-6: Sets and dictionaries
- Chapter-7: File handling
- Chapter-8: Object Oriented Programming

How to read these lessons?

- Do not trust any piece of code blindly.
- Execute the code and observe the output.
- Think about the output.
- Verify if the explanation given in the text matches your observations.

Code does not lie. Learners should run every snippet.

Python Version

Lessons use Python 3.8 or higher. Python 2 users are strongly encouraged to switch, since Python 2 has reached end of life.

Setting up Replit

Replit is an online coding environment, ideal for learning, and it is used extensively in the course. Sign up at replit.com using your Online Degree account and use Replit's tutorial to get started.

Installing Python on your System

For a local installation, see python.org/downloads and follow a step-by-step guide. Having Python installed is useful for Diploma-level subjects.

History

Python first appeared in February 1991, created as a hobby project by Dutch programmer Guido van Rossum, who served as *benevolent dictator for life* until stepping down in 2018. On the naming, the official documentation notes that van Rossum was reading scripts from "Monty Python's Flying Circus," a 1970s BBC comedy, and wanted a name that was "short,

unique, and slightly mysterious." Python is now over 30 years old, built by people and used by thousands worldwide — jump in with an open mind.

NOTE

An image caption in the source reads: "Guido Van Rossum at the Dropbox Headquarters in 2014."

Explore

1. Check out the Python Software Foundation website to learn about the organization behind Python.
2. Read a Dropbox blog interview with Guido van Rossum (trivia: he worked at Dropbox for six and a half years).
3. Watch documentaries and interviews where Guido discusses Python's origins, for a humanistic view of technology.
4. StackOverflow, a question-and-answer forum for programming, will likely become heavily used — but use it wisely, and avoid using it for assignment answers.
5. Look into the official documentation on the Python website.

1

Introduction to Python

Lesson 1.1 — Python Shell, Output, Literals and Data Types

Python shell | Replit Console

This lesson works with the Python interpreter in *Interactive Mode*, often called the Python shell. It executes individual lines of code and shows the output immediately. Throughout the lessons it is referred to simply as "the interpreter." In Replit this corresponds to the console screen on the right of the repl.

Prompts

The orange symbol displayed is called a *prompt*. Its role is similar to a blinking cursor while editing documents — it is "an invitation to type code." Code typed at the prompt is executed by the interpreter, represented by `>>>`.

```
PYTHON REPL
>>> print('Hello World!')
Hello World!
```

Output

`print()` is a built-in function. A function accepts inputs and returns outputs, and "built-in" means Python readily provides it.

```
PYTHON REPL
>>> print('Hello World!')
Hello World!
>>> print("Hello World!")
Hello World!
```

The text enclosed in parentheses is a **string** — "a sequence of characters enclosed in quotes." Strings use single or double quotes, but a single quote cannot be matched with a double quote. This is useful for apostrophes:

```
PYTHON REPL
>>> print("India's capital is New Delhi.")
```

`print()` can also print numbers:

PYTHON REPL

```
>>> print(1)
1
>>> print(2.0)
2.0
```

Multiple items can appear on one line:

PYTHON REPL

```
>>> print(1, 2)
1 2
>>> print('online', 'degree', 'program')
online degree program
```

`print` separates multiple values with a delimiter, which is a space by default. Calling `print` with no input prints a blank line:

PYTHON REPL

```
>>> print()

>>>
```

Using `print` without parentheses:

PYTHON REPL

```
>>> print
<built-in function print>
```

But this raises an error:

PYTHON REPL

```
>>> print 'Hello World!'
File "<stdin>", line 1
    print 'Hello World!'
      ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean
print('Hello World!')?
```

Programming languages have strict syntaxes that must be followed exactly. Parentheses execute the function and pass values (arguments) to it.

Emojis

```
PYTHON REPL
>>> print('\N{smiling face with smiling eyes}')
😊
>>> print('\N{grinning face}')
😄
>>> print('\N{smiling face with halo}')
👀
>>> print('\N{thinking face}')
🤔
```

A full list of emojis is available in the Unicode emoji charts.

Literals and Variables

Strings like `'Hello World!'` and numbers like `1` and `2.0` are *literals* — "a literal is something that describes a constant value." Variables are containers for storing values:

```
PYTHON REPL
>>> x = 1
>>> print(x)
1
>>> y = 'a string'
>>> print(y)
a string
>>> foo_bar = 123.456
>>> print(foo_bar)
123.456
```

`=` is the assignment operator, used to define a new variable or to update an existing one:

```
PYTHON REPL
>>> x = 1          # define a new variable
>>> x = x + 1      # update an existing variable
>>> print(x)
2
```

The assignment operator "is evaluated from right to left" — the right-side expression evaluates first, then the result is assigned to the left variable.

Basic Data Types | `type()`

The basic data types covered are Integer, Float, String, and Boolean.

Integer

The `int` type represents integers. `type` determines an object's type:

```
PYTHON REPL
>>> print(1)
1
>>> type(1)
<class 'int'>
```

Float

The `float` type represents real numbers:

```
PYTHON REPL
>>> print(1.0)
1.0
>>> type(1.0)
<class 'float'>
```

A valid float literal:

```
PYTHON REPL
>>> print(1.)
1.0
```

`1.` and `1.0` are the same literal.

String

The `str` type represents strings:

```
PYTHON REPL
>>> print('one')
one
>>> type("one")
<class 'str'>
```

Boolean

The `bool` type represents boolean values:

```
PYTHON REPL
>>> print(True)
True
>>> type(False)
<class 'bool'>
```

NOTE

`bool` values are case sensitive — `true` and `false` are not `bool` values.

Comments

A comment is "a line of text that is not executed by the interpreter." Comments begin with `#`:

```
PYTHON REPL
>>> # This is a comment
>>> # print(1)
>>>
```

Comments can also appear at the end of a line of code:

```
PYTHON REPL
>>> print(1) # This line is printing the value 1
1
```

Adding comments is one way to make code more readable.

Lesson 1.2 — Operators and Expressions

Operators

Arithmetic Operators

Python's arithmetic operators are all binary (they take two operands):

OPERATOR	OPERATION
<code>+</code>	Addition
<code>-</code>	Subtraction
<code>*</code>	Multiplication
<code>/</code>	Division
<code>//</code>	Floor division
<code>%</code>	Modulus
<code>**</code>	Exponentiation

Interactive examples show `10 + 5` gives `15`, `10 / 5` gives `2.0`, `10 // 5` gives `2`, `10 % 5` gives `0`, and `10 ** 5` gives `100000`.

Key notes: `//` gives the quotient (`8 // 3` is `2`), `%` gives the remainder (`10 % 3` is `1`), and `**` returns `x` to the power `y`. `/` and `//` differ — `5 / 2` gives `2.5` while `5 // 2` gives `2`.

There are also **unary** `+` and `-` (one operand): `- 2` gives `-2`, `+ 2` gives `2`. Context decides the meaning; for example `1 - - 1` gives `2` (subtraction on the left, unary minus on the right). Operands can also be variables.

Relational Operators

Binary comparison operators return boolean values:

OPERATOR	OPERATION
<code>></code>	greater than
<code><</code>	less than
<code>>=</code>	greater than or equal to
<code><=</code>	less than or equal to
<code>==</code>	double equal to
<code>!=</code>	not equal to

Results are `True` / `False` and can be assigned to variables.

WARNING

Do not confuse `==` (equality) with `=` (assignment).

Logical Operators

OPERATOR	OPERATION
<code>not</code>	negation
<code>and</code>	logical conjunction
<code>or</code>	logical disjunction

`and` and `or` are binary; `not` is unary. Parentheses after `not` are optional (`not x` or `not(x)`).

CONVENTION

Both `1 + 2` and `1+2` give `3` . The course uses spaces around operators, including `=` (`x = 2` , not `x=2`), though both forms are valid.

Operator Chaining

Python allows chaining, so `10 < 11 <= 12` is `True` , equivalent to `10 < 11 and 11 <= 12` .

Expressions

An expression is "some combination of literals, variables and operators." Examples include `1 + 4 / 4 ** 0` and `not True and False` . Each evaluates to a typed value. Two types are studied:

- **Arithmetic:** type `int` or `float`
- **Boolean:** type `bool`

Types of Expressions

Arithmetic Expressions

```
PYTHON REPL
>>> type(1 + 2)
<class 'int'>
>>> type(1.0 + 2)
<class 'float'>
```

The conclusion is that `float` is more dominant than `int` irrespective of the operator involved — all operations mixing a float yield a float.

Boolean Expressions

Relational and logical expressions yield `bool`:

```
PYTHON REPL
>>> type(2 > 1)
<class 'bool'>
>>> type(True and False)
<class 'bool'>
```

Truth table for `X or Y`:

X	Y	X OR Y
True	True	True
True	False	True
False	True	True
False	False	False

Lesson 1.3 — Arithmetic and Boolean Expressions

Arithmetic Expressions

Precedence

Consider an arithmetic expression that uses multiple operators:

```
PYTHON REPL
>>> 4 // 2 - 1
1
```

This can be interpreted two ways: `(4 // 2) - 1 = 2 - 1 = 1` or `4 // (2 - 1) = 4 // 1 = 4`. Python follows the first. When an expression mixes operators, the interpreter must decide how to parenthesize it — that is, which operator takes **precedence**. From this example, floor division (`//`) has greater precedence than subtraction (`-`).

The lesson references a precedence table where higher-precedence operators appear at the top, and operators sharing a cell have equal precedence (for instance, `+` and `-`).

PYTHON REPL

```
>>> 3 ** 2 * 4 - 4
32
```

The parenthesization proceeds as $(3 ** 2) * 4 - 4$, then $((3 ** 2) * 4) - 4$, evaluating to $((3 ** 2) * 4) - 4 = (9 * 4) - 4 = 36 - 4 = 32$.

Order

PYTHON REPL

```
>>> 3 - 2 + 1
2
```

This could mean $(3 - 2) + 1 = 1 + 1 = 2$ or $3 - (2 + 1) = 3 - 3 = 0$. Python uses the first. But this does not mean subtraction outranks addition — they share precedence. Python evaluates expressions from **left to right**. The two exceptions are `**` and `=`, which evaluate right to left.

PYTHON REPL

```
>>> 4 - 3 - 1
0
```

The options are $(4 - 3) - 1 = 1 - 1 = 0$ or $4 - (3 - 1) = 4 - 2 = 2$. The first (left-to-right) is used.

PYTHON REPL

```
>>> 8 % 4 % 2
0
```

Readers are asked to test which parenthesization matches — this is left as an exercise:

PYTHON REPL

```
>>> (8 % 4) % 2
>>> 8 % (4 % 2)
```

Finally, `**` behaves uniquely:

PYTHON REPL

```
>>> 2 ** 3 ** 0
2
```

The options are $(2 ** 3) ** 0$ or $2 ** (3 ** 0)$. Python follows the second (right to left). This right-to-left behavior applies only to exponentiation and assignment.

Boolean expressions

The simplest boolean-producing expression:

```
PYTHON REPL
>>> 1 > 0
True
>>> type(1 > 0)
<class 'bool'>
```

Expressing that 3.14 lies between 3 and 4:

```
PYTHON REPL
>>> 3 < 3.14 and 4 > 3.14
True
```

This can also be written as:

```
PYTHON REPL
>>> 3 < 3.14 < 4
True
```

Adding boolean literals:

```
PYTHON REPL
>>> 10 > 20 or True
True
```

As an exercise:

```
PYTHON REPL
>>> False or False or False or False or True
```

Precedence and Order

Logical operators also have precedence and evaluate left to right.

```
PYTHON REPL
>>> not True and False
False
```

The two parenthesizations are `not(True) and False = False and False = False` or `not(True and False) = not(False) = True`. Python follows the first, consistent with logical precedence rules.

```
>>> True or False and False
True
```

Beware of float !

[illegible]

```
>>> 0.1 ** 100 == 0.0
False
>>> 0.1 ** 1000 == 0.0
True
```

```
>>> 0.1 * 3 == 0.3
False
```

```
>>> 0.1 * 3
0.30000000000000004
```

14

Short Circuit Evaluation

```
PYTHON REPL
>>> 1 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

Division by zero produces an error, as expected. But the next result is surprising:

```
PYTHON REPL
>>> True or (1 / 0)
True
```

No error appears. The explanation: evaluation runs left to right with `or`. Since the left operand is `True`, the whole expression is `True` regardless of the right operand, so the interpreter skips evaluating it. "This behaviour is called short circuit evaluation."

A more complex example:

```
PYTHON REPL
>>> (not((3 > 2) or (5 / 0))) and (10 / 0)
False
```

The lesson references a flow chart with arrows showing evaluation order. Following them reveals that the two problematic expressions — `5 / 0` and `10 / 0` — are never evaluated.

Lesson 1.4 — The Replit Editor, Errors and Debugging

Replit Editor

This section explains moving from Replit's console to the editor. The key advantage is that code is automatically saved — described as being like "Google Docs for code." The editor is to the left of the console. After typing code, you click the green `Run` button at the top, and output appears in the console. From this point on, the book drops the prompt symbol before each line.

Errors

Introduction

Enter and run this code:

```
PYTHON
print('123')
```

OUTPUT

```
File "main.py", line 1
  print('123)
        ^
SyntaxError: EOL while scanning string literal
```

The first lines are an error message — the interpreter's warning that something is wrong. Here it is a `SyntaxError`. The details explain:

- **EOL** stands for **End Of Line**
- scanning a string literal

The `^` sign points to where the error occurred, acting as a visual aid. The problem is the missing ending quote `'`. The fix:

PYTHON

```
print('123')
```

Debugging

Errors in code are also called bugs, and fixing them is called **debugging**. The process works as follows:

1. **Run**: run the code
2. **Detect**: the interpreter intimates the coder of the error
3. **Understand**: the coder has to understand the error message and go back to the code to see what went wrong
4. **Update**: fix the error by modifying or updating one or more lines of the code

Afterward the coder runs the code again, closing the loop. If another error appears, the process repeats. With large codebases, fixing bugs "might take several hours or even days."

Exceptions

These are not syntax errors. First example:

PYTHON

```
1 / 0
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    1 / 0
ZeroDivisionError: division by zero
```

This is a `ZeroDivisionError`. There is no syntax problem here. Errors detected during a program's execution are called **exceptions**. Another example:

PYTHON

```
1 + 'one'
```


OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    1 + 'one'
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

This is a `TypeError` — an integer and a string cannot be added with the `+` operator.

Next is `NameError`:

PYTHON

```
print('There is no problem with this line')
print(x ** 2)
```

OUTPUT

```
There is no problem with this line
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(x ** 2)
NameError: name 'x' is not defined
```

The first line prints correctly because the interpreter executes code top to bottom. A `NameError` occurs when referencing a variable that has not been defined. Referencing variables is covered in the next chapter.

Wrong Code Snippets

Incorrect code examples are demonstrated this way:

PYTHON

```
##### Alarm! Wrong code snippet! #####

# Incorrect code will go here #

##### Alarm! Wrong code snippet! #####
```

Lesson 1.5 — Strings: Quotes, Length, Operations, Escapes

Quotes: single, double and triple

A string is any sequence of characters wrapped in single or double quotes. Example strings include:

PYTHON

```
"this is a string"
'this is also a string'
'1 + 1 = 2'
"! , ? , _ , @ are special characters"
"if you need to use apostrophe ('), you can use double quotes"
```

Be consistent in choosing single or double quotes. Python also supports triple quotes (`'''`), which are especially useful for multi-line strings. To capture these three lines in one string:

```
OUTPUT
first line
second line
third line
```

This code produces a `SyntaxError`:

PYTHON

```
x = 'first line
second line
third line'
print(x)
```

Triple quotes solve this:

PYTHON

```
x = '''first line
second line
third line'''
print(x)
```

Typing `x` in the console after running gives:

```
OUTPUT
'first line\nsecond line\nthird line'
```

The `\n` is a newline character (see the escape characters section).

Length

Length is the number of characters in a string, found with the built-in `len` function:

PYTHON

```
x = 'good'
print(len(x))
```

This outputs `4`. Unlike C, Python has no separate character type — "A character in Python is represented by a string of length 1." Examples of length-1 strings:

PYTHON

```
x = 'a'
y = 'b'
```

Empty strings are also allowed:

PYTHON

```
x = ''
print(len(x))
```

The empty string has length 0.

Operations on strings

Concatenation

Concatenation joins strings using the `+` operator — "just a fancy term for joining two strings together":

PYTHON

```
string1 = 'first'
string2 = ','
string3 = 'second'
string4 = string1 + string2 + string3
print(string4)
```

OUTPUT

```
first,second
```

Replication

The `*` operator makes and joins multiple copies of a string:

PYTHON

```
s = 'good'
five_s = s * 5
print(five_s)
```

OUTPUT

```
goodgoodgoodgoodgood
```

This demonstrates the adage "multiplication is repeated addition":

PYTHON

```
s = 'good'
s * 5 == s + s + s + s + s # This expression evaluates to True
```

Comparison

Strings can be compared, starting with the `==` operator:

PYTHON

```
x = 'python'
print(x == 'python', x == 'nohtyp')
```

OUTPUT

```
True False
```

Two strings are equal "if and only if both of them represent exactly the same sequence of characters." Next:

PYTHON

```
print('good' > 'bad')
print('nine' < 'one' )
print('a' < 'ab' < 'abc' < 'b')
```

OUTPUT

```
True
True
True
```

These examples show that length is not the comparison metric; instead Python uses alphabetical ordering, more precisely **lexicographic ordering**.

LEXICOGRAPHIC ORDERING

"The first characters from the two strings are compared. If they differ this determines the outcome of the comparison. If they are equal, then the second character of both the strings are compared. This process continues until either string is exhausted."

For comparing characters, "Python's string type uses the Unicode standard for representing characters." Unicode is a specification listing every character used by human languages, giving each a unique code. Characters are represented by **code points**, and a code point value is an integer used for ordering.

The built-in `ord` function returns a character's code point:

PYTHON

```
print(ord('a'), ord('b'))
print(ord('a'), ord('A'))
```

OUTPUT

```
97 98
97 65
```

This shows why `'a' < 'b'` is `True` (97 < 98), and lets us infer `'A' < 'a'` returns `True`.

Escape characters

The backslash `\` is the escape character, used partly to represent whitespace like tabs and newlines:

PYTHON

```
print('This is the first sentence.\nThis is the second sentence.')
```

OUTPUT

```
This is the first sentence.
This is the second sentence.
```

`\n` is a newline character. Though it looks like two characters, "`\n`" is still regarded as a single character." Verify with:

PYTHON

```
x = '\n'
print(len(x))
```

This outputs `1`. The tab character is `\t`:

PYTHON

```
print('a\tb')
```

OUTPUT

```
a    b
```

Quotes can be escaped with `\'`, useful for apostrophes in single-quoted strings:

PYTHON

```
print('India\'s capital is New Delhi')
```

OUTPUT

```
India's capital is New Delhi
```

Remove the backslash and predict why an error occurs.

Substrings

A substring is a string contained within another — for example `'good'` is a substring of `'very good'`, but `'very good'` is not a substring of `'verygood'`. The `in` keyword checks for substrings:

PYTHON

```
a = 'good'
b = 'very good'
present = a in b
print(present)
not_present = b in a
print(not_present)
```

OUTPUT

```
True
False
```

`in` is "a powerful keyword which has several other uses" and can combine with `not`:

PYTHON

```
a = 'abc'
b = 'ab'
print(a not in b)
```

OUTPUT

True

Lesson 1.6 — Strings: Indexing, Slicing, Immutability, Methods

This lesson recaps prior string operations and introduces the sequential nature of strings, serving as a lead-in to lists, along with string methods. The earlier operations covered include finding length with `len`, concatenation with `+`, replication with `*`, comparison using relational operators like `>`, `<`, `==`, and use of the `in` keyword.

Indexing

A string is a sequence of characters, and sequences support indexing. Using "world" as an example, 'w' is the first letter, 'o' the second, and so on. The index formally denotes an element's position, and computer science widely uses zero-based numbering starting from 0.

PYTHON

```
word = 'world'
print(word[0])
print(word[1])
print(word[2])
print(word[3])
print(word[4])
```

OUTPUT

w
o
r
l
d

`word[i]` gives the character at index `i`, informally the letter at position `i + 1`.

PYTHON

```
word = 'world'
print(word[5])
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(word[5])
IndexError: string index out of range
```

The interpreter throws an `IndexError` because index 5 is out of range; the last character is at index 4.

PYTHON

```
word = 'world'
print(word[-1])
```

OUTPUT

```
d
```

Python supports negative indexing. The lesson offers an analogy: descending a staircase where, upon reaching the last step, "some invisible hand magically transports you back to the top most step." This is likened to the Penrose stairs. An index of `-1` points to the last element, moving backward to `-5` for the first.

PYTHON

```
word = 'world'
print(word[-1])
# ... please add the remaining lines!
print(word[-5])
```

Unlike the Penrose stairs, this cannot repeat forever — `print(word[-6])` throws an `IndexError`.

Slicing

Slicing extracts a substring using the slice operator. The lesson uses an example of IIT-M email ids of the form:

branch_year_number@iitm.ac.in

Each branch has a two-letter code (`CS` for Computer Science, `ME` for Mechanical Engineering), followed by a two-digit joining year, then a three-digit roll number. Sample email ids:

OUTPUT

```
CS_10_014@iitm.ac.in
ME_11_123@iitm.ac.in
BT_17_001@iitm.ac.in
```

PYTHON

```
email = 'CS_10_014@iitm.ac.in'
roll = email[6 : 9]
print(roll)
```

The `start:stop` operator is "our knife in slicing sequences." To extract `014` at indices 6, 7, 8, you start at index 6 and stop before index 9 — the character at `stop` is excluded.

PYTHON

```
email = 'CS_10_014@iitm.ac.in'
branch = email[0 : 2]
year = email[3 : 5]
roll = email[6 : 9]
college = email[10 : 14]
# Print each one of them and check the output
```

PYTHON

```
email = 'CS_10_014@iitm.ac.in'
in_roll = email[ : 9]
print(in_roll)
```

This outputs `CS_10_014`. When no start is given, it defaults to 0; when no stop is given, it defaults to the end of the string, i.e. `len(email)`.

PYTHON

```
email = 'CS_10_014@iitm.ac.in'
domain = email[-10 : ]
print(domain)
```

This outputs `iitm.ac.in`, combining negative indexing and slicing.

PYTHON

```
word = 'world'
print(word[-4 : 3])
print(word[1 : -2])
```

Immutability

PYTHON

```
word = 'some string'
word[0] = 'S'
```

The interpreter throws a `TypeError` with the message `'str' object does not support item assignment`. An object is mutable if it can be changed, immutable if it cannot. Strings are **immutable**, so characters cannot be modified **in-place**.

This differs from reassignment:

PYTHON

```
word = 'some string'
word = 'Some string'
```

Here, `word` is assigned an entirely new string literal rather than modified in place. There are two distinct literals — `'some string'` and `'Some string'` — and the former is not transformed into the latter. Mutable and immutable objects are explored further in chapter 5.

Methods

The problem posed: accept a sentence and output it with the first letter capitalized. For input `'this is a chair.'`, the output should be `'This is a chair.'`.

PYTHON

```
sentence = input()
cap_sentence = sentence.capitalize()
print(cap_sentence)
```

`capitalize` is a *method* — a function defined for a specific object (here, the `str` type) and called using that object. Calling it on an `int` produces an error:

PYTHON

```
##### Alarm! Wrong code snippet!
a = 1
a.capitalize()
##### Alarm! Wrong code snippet!
```

`sentence.capitalize()` returns a string, which is assigned to `cap_sentence`. Another problem: check whether a string is a valid person's name. Assuming a name has only alphabets without special characters or numbers, the `isalpha` method checks this:

PYTHON

```
# name is some pre-defined string
valid = name.isalpha()
print(valid)
```

`name.isalpha()` returns a boolean — `True` if every character is alphabetic and the string is non-empty, `False` otherwise. A comprehensive list of string methods is available in the Python documentation (docs.python.org, Standard Types — String Methods).

2

Conditionals

Lesson 2.1 — Variables, Assignment and Dynamic Typing

Introduction

Variables are containers used to store values. In Python, they are defined using the assignment operator `=`.

PYTHON

```
x = 1
y = 100.
z = "good"
```

Variables can also be updated with the assignment operator:

PYTHON

```
x = 1
print('The initial value of x is', x)
x = 2
print('The value after updating x is', x)
```

OUTPUT

```
The initial value of x is 1
The value after updating x is 2
```

Assignment Operator

The syntax of an assignment statement is:

PYTHON

```
variable_name = expression
```

The operator works right to left — the right-hand expression is evaluated first, then its value is assigned to the left-hand variable.

PYTHON

```
x = 1 + 2 * 3 / 2
print(x)
```

OUTPUT

```
4.0
```

Placing a literal on the left of the operator causes an error:

PYTHON

```
##### Alarm! Wrong code snippet! #####  
3 = x  
##### Alarm! Wrong code snippet! #####
```

OUTPUT

```
SyntaxError: cannot assign to literal
```

The assignment statement binds the name on the left to an object on the right. Consider a sequence where, after line 1, `x` binds to the `int` `8`; a trailing period on line 2 makes `10` a `float`, so `x` holds `18.0`. On the next line `y = x` makes both names bind to the same object. When `x` is later reassigned, it binds to a new object, but `y` remains bound to `18.0`.

Do not confuse assignment with equality:

PYTHON

```
x = 2 # this is the assignment operator  
x == 2 # this is the equality operator
```

Assignment creates or updates variables; equality compares two expressions. They are not interchangeable.

DYNAMIC TYPING

Python is dynamically typed. A variable is "a value bound to a name" where the value has a type but the variable itself does not.

PYTHON

```
a = 1  
print(type(a))  
a = 1 / 2  
print(type(a))  
a = "IIT Madras"  
print(type(a))
```

OUTPUT

```
<class 'int'>  
<class 'float'>  
<class 'str'>
```

Here `a` starts as an `int`, becomes a `float`, and then a `str`.

Referencing versus Defining

Using an already-defined variable in an expression is called referencing.

PYTHON

```
x = 2  
print(x * x, 'is the square of', x)
```

The second line references `x` from the first. Referencing a variable before assigning it raises a `NameError`:

PYTHON

```
print(someVar)
```

OUTPUT

```
NameError: name 'someVar' is not defined
```

Keywords and Naming Rules

Keywords are reserved words with special meaning. Some of them are:

PYTHON

```
not, and, or, if, for, while, in, is, def, class
```

We have already seen `not`, `and`, `or`; the rest appear in later chapters. Keywords cannot be variable names:

PYTHON

```
##### Alarm! Wrong code snippet! #####
and = 2
##### Alarm! Wrong code snippet! #####
```

Other naming rules:

- A variable name may only contain alphanumeric characters and underscores:

- a – z
- A – Z
- 0 – 9
- `_`

- A name must start with a letter or underscore.

Observations that follow:

- A name cannot start with a number.
- Names are case-sensitive (`age`, `Age`, `AGE` are three different variables).

These are rules, not conventions — violating them raises a `SyntaxError`.

PYTHON

```
##### Alarm! Wrong code snippet! #####
3a = 1
##### Alarm! Wrong code snippet! #####
```

OUTPUT

```
SyntaxError: invalid decimal literal
```

Reusing Variables

Variables can help compute other variables — common in programming and data science. Given the equations $y = x^2$ and $z = (x+1)(y+1)$, evaluated at $x = 10$:

PYTHON

```
x = 10
y = x ** 2
z = (x + 1) * (y + 1)
```

Multiple Assignment

Consider defining two variables:

PYTHON

```
x = 1
y = 2
```

Python allows a compact single-line form. This assigns 1 to `x` and 2 to `y`:

PYTHON

```
x, y = 1, 2
```

Order matters — this assigns 2 to `x` and 1 to `y`:

PYTHON

```
x, y = 2, 1
```

This works through tuple packing and unpacking (covered in Chapter 5). You can also initialize several variables to the same value:

PYTHON

```
x = y = z = 10
print(x, y, z)
```

OUTPUT

```
10 10 10
```

The equality breaks as soon as any one is updated:

PYTHON

```
x = x * 1
y = y * 2
z = z * 3
print(x, y, z)
```

OUTPUT

```
10 20 30
```

Assignment Shortcuts

Run this and observe the output:

PYTHON

```
x = 1
x += 1
print(x)
```

`+=` is new: `x += a` increments `x` by `a` — it adds `a` to `x` and stores the result in `x`. It is equivalent to `x = x + a`. This applies to other operators too:

SHORTCUT**MEANING**`x += a``x = x + a``x -= a``x = x - a``x *= a``x = x * a``x /= a``x = x / a``x %= a``x = x % a``x **= a``x = x ** a`

The arithmetic operator must come before the assignment operator. Swapping will not work:

PYTHON

```
x = 1
x += 1
print(x)
```

This outputs `1` because `+` is treated as a unary operator. Statements like `x =* 1` or `x =/ 2` cause errors.

Deleting Variables

Variables can be removed with the `del` keyword:

PYTHON

```
x = 100
print('x is a variable whose value is', x)
print('we are now going to delete x')
del x
print(x)
```

The last line raises a `NameError` because `x` was deleted, so accessing it afterward fails.

NOTE

Python is both dynamically typed and strongly typed.

Lesson 2.2 — Input, Type Conversion and Built-in Functions

Input

This lesson covers accepting user input, which is a routine part of programming. Any software given to a customer needs a functional interface for user interaction. Apps like Facebook, Instagram, and Twitter regularly accept input, even though we rarely view it through a programming lens. For instance, when commenting on a Facebook post, the comment text is the input, and backend code processes it and displays it attractively.

Python's built-in `input()` function accepts user input — "a simple yet powerful function":

PYTHON

```
x = input()
print('The input entered by the user is', x)
```

When run, the interpreter waits for text in the console. Pressing Enter signals that input is complete, and the text is stored in `x`.

PYTHON REPL

```
1
The input entered by the user is 1
```

You can prompt the user by passing an instruction as an argument:

PYTHON

```
x = input('Enter an integer between 0 and 10: ')
print('The number entered by the user is', x)
```

To inspect the variable's type:

PYTHON

```
x = input()
print('The input entered by the user is of type', type(x))
```

Try `int`, `float`, `str`, and `bool` inputs. The key point: `input()` always returns a string. Even entering `123` is processed as the string `'123'`. To accept an integer, we use type conversion.

Type Conversion

To convert a string into an integer, Python provides `int()`:

PYTHON

```
x = '123'
print('The type of x is', type(x))
y = int(x)
print('The type of y is', type(y))
```

The third line converts an object of type `str` into type `int`. The reverse works too, using `str()`:

PYTHON

```
x = 123
print('The type of x is', type(x))
y = str(x)
print('The type of y is', type(y))
```

To accept an integer, take a string first and convert it:

PYTHON

```
x = input('Enter an integer: ')
x = int(x)
print('The integer entered by the user is', x)
```

This can be shortened to a single line:

PYTHON

```
x = int(input())
print('The integer entered by the user is', x)
```

The first line composes two functions — passing the output of the inner `input()` as input to the outer `int()`. If a float value is entered:

PYTHON

```
x = int(input())    # user enters a float value here
```

This throws a `ValueError`. For example, `int('1.23')` fails because the value in quotes is a float, not an int, so it cannot be converted.

Built-in Functions

The term **built-in functions** refers to functions that are already defined. Loosely, a function in Python is an object that takes inputs and produces outputs — for example, `print()` accepts input and prints it to the console.

A few more useful functions:

- `round()` takes a number and returns the nearest integer. `round(1.2)` returns `1`, `round(1.9)` returns `2`.
- `abs()` returns the absolute value. `abs(-1.2)` returns `1.2`.
- `int()` is more involved. An integer in a string returns that integer: `int('123')` is `123`. A float has its decimal part discarded: `int(1.2)` returns `1` and `int(-2.5)` returns `-2`. A float passed as a string throws a `ValueError`. `int('2.5')` gives:

OUTPUT

```
ValueError: invalid literal for int() with base 10: '2.5'
```

- `pow()` : `pow(x, y)` returns `x` raised to the power `y`, the same as the `**` operator. Generally `**` is faster, but the difference is negligible for small numbers, and `pow()` improves readability. With a third argument, `pow(x, y, z)` returns `x^y mod z` — the remainder when `x^y` is divided by `z`.

- `isinstance()` checks whether an object is of a specified type. `isinstance(3, int)` returns `True` since `3` is an `int`. The first argument can be any object; if `x` is a `str`, then `isinstance(x, str)` returns `True`.

The Python documentation (docs.python.org, Built-in Functions) provides an exhaustive list of built-in functions.

Lesson 2.3 — Conditional Statements

This lesson introduces conditional statements using a motivating problem: accept an integer, then print `positive` if it is greater than zero, `negative` if less than zero, or `zero` otherwise. Conditional statements are a fundamental building block in computer science that allow code to execute conditionally.

if statement

A simpler version first: accept an integer and print `non-negative` if it is greater than zero. The `if` keyword is followed by a boolean expression called the **condition**. If the condition is `True`, the indented body runs; otherwise it is skipped.

PYTHON

```
x = int(input())
if x >= 0:
    print('non-negative')
```

Indentation matters and must be consistent — the first level uses four spaces. Consider two examples of the **if-block**.

Positive `x`:

PYTHON

```
# Left
x = 1
if x >= 0:
    print('non-negative')
    print('inside if')
print('outside if')
```

Since the condition is `True`, the indented lines run:

OUTPUT

```
non-negative
inside if
outside if
```

Negative `x`:

PYTHON

```
# Right
x = -1
if x >= 0:
    print('non-negative')
    print('inside if')
print('outside if')
```

Here the condition is `False`, so the body is skipped:

```
OUTPUT
outside if
```

if-else

Next problem: print `non-negative` if the number is $\geq$ zero, else print `negative`. The `else` keyword runs its body when the if-condition is `False`.

PYTHON

```
x = int(input())
if x >= 0:
    print('non-negative')
else:
    print('negative')
```

Points to remember:

- `if` and `else` are at the same indentation level.
- `else` can never occur independently of an `if`.
- `else` cannot have a new condition attached.

Example of **wrong code** demonstrating the last two points:

PYTHON

```
##### Alarm! Wrong code snippet! #####
else:
    print(1)
##### Alarm! Wrong code snippet! #####

##### Alarm! Wrong code snippet! #####
x, y = 1, 2
if x >= y:
    print(1)
else x < y:
    print(1)
##### Alarm! Wrong code snippet! #####
```

if-elif-else

The `elif` keyword is shorthand for else-if, solving the original problem:

PYTHON

```

x = int(input())
if x > 0:
    print('positive')
elif x == 0:
    print('zero')
else:
    print('negative')
# End of code

```

Table of inputs and outputs:

INPUT	OUTPUT
x = 1	positive
x = 0	zero
x = -1	negative

The block has three sub-blocks: the if-block, the elif-block, and the else-block. The interpreter checks the if-condition first; if `True`, its body runs, then control jumps past the block. If `False`, it checks the elif; if that is `True`, its body runs then jumps past the block. If the elif is `False`, the else runs.

General syntax:

PYTHON

```

if <condition_1>:
    <statement_1>
elif <condition_2>:
    <statement_2>
else:
    <statement_3>

```

Features to note:

- Exactly one of the three statements runs.
- Once an `if` or `elif` condition is `True`, its body runs and flow exits the whole block.
- There can be multiple `elif` conditions after the `if`.
- An `else` cannot come before an `elif`; the final `else` is optional but must come last if present.

Nested conditional statements

Problem: accept three distinct integers; print `in ascending order` if entered in ascending order, else `not in ascending order`.

An **incomplete** solution:

PYTHON

```
# Incomplete solution
x = int(input())
y = int(input())
z = int(input())
if x < y:
    print('in ascending order')
else:
    print('not in ascending order')
```

This fails to check whether `y < z`, so an input like `x, y, z = 1, 3, 2` wrongly prints `in ascending order`. The complete solution:

PYTHON

```
x = int(input())
y = int(input())
z = int(input())
if x < y:
    if y < z:
        print('in ascending order')
    else:
        print('not in ascending order')
else:
    print('not in ascending order')
```

Each new if block's body must be indented one level relative to its condition. Placing one conditional inside another is called **nesting**: the outer block contains the inner block. Each `else` pairs with the `if` at the same indentation level.

Defining variables inside if

Consider this code:

PYTHON

```
x = int(input())
if x % 5 == 0:
    output = 'the number is divisible by 5'
print(output)
```

Running it with different inputs reveals: when the input is a multiple of 5, it runs fine; otherwise it raises a `NameError`. This happens because the variable `output` is only created if the assignment line executes at run-time — its presence in the source code alone is not enough to define it.

Lesson 2.4 — System Libraries: calendar, time, this

Library

A library is a collection of functions sharing a common theme. This is a loose definition that becomes clearer once you begin working with a library directly.

calendar

Consider this problem: in the year 3000, on which day of the week will the 15th of August fall? Python solves it easily:

PYTHON

```
import calendar
calendar.prmonth(3000, 8)
```

OUTPUT

```
August 3000
Mo Tu We Th Fr Sa Su
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

The 15th falls on a Friday — accomplished in just two lines. `calendar` is one of many libraries in Python's standard library. Here, `calendar` names the library and `import` is the keyword used to bring it into the code.

`calendar` groups together functions related to calendars, and `prmonth()` is one of them. It takes `<year>` and `<month>` and displays that month's calendar. You must import the library before using its functions. Here is what happens if you skip that step:

PYTHON

```
# import calendar
calendar.prmonth(3000, 8)
```

OUTPUT

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
NameError: name 'calendar' is not defined
```

To reach a function inside a library, use this syntax:

PYTHON

```
<calendar>.<function>(<arguments>)
```

Another approach uses the `weekday` function:

PYTHON

```
import calendar
print(calendar.weekday(3000, 8, 15))
```

The output is `4`. Days map to numbers as follows:

DAY	NUMBER
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5
Sunday	6

time

Now a hypothetical scenario: you are stranded on an island in the Indian Ocean with a device running only a Python interpreter, and you want the current date and time.

PYTHON

```
from time import ctime
print('The current time is:', ctime())
```

OUTPUT

```
The current time is: Fri Apr 2 12:24:43 2021
```

The import statement on the first line looks different because `from` is a new keyword. That line essentially says: from the `time` library, import the `ctime` function. This style is handy when you need only one or two functions from a library.

PYTHON

```
from time import ctime, sleep
print('Current time is:', ctime())
print('I am going to sleep for 10 seconds')
sleep(10)
print('Current time is:', ctime())
```

`sleep(x)` is a `time` function that pauses program execution for `x` seconds. If you plan to use many functions from a library, it is a poor idea to import each one separately — better to import the whole library instead.

this

As a fun exercise, try:

PYTHON

```
import this
```

OUTPUT

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.  
Special cases aren't special enough to break the rules.  
Although practicality beats purity.  
Errors should never pass silently.  
Unless explicitly silenced.  
In the face of ambiguity, refuse the temptation to guess.  
There should be one-- and preferably only one --obvious way to do it.  
Although that way may not be obvious at first unless you're Dutch.  
Now is better than never.  
Although never is often better than *right* now.  
If the implementation is hard to explain, it's a bad idea.  
If the implementation is easy to explain, it may be a good idea.  
Namespaces are one honking great idea -- let's do more of those!
```

These are nuggets of wisdom from Tim Peters, a major contributor to the Python programming language. Some points, like "readability counts," make immediate sense.

3

Loops

Lesson 3.1 — The while Loop; break and continue

Introduction

How do we sum the first five positive integers? A simple direct approach:

PYTHON

```
print(1 + 2 + 3 + 4 + 5)
```

Now consider summing the first 1,000,000 integers — the manual approach is impractical. An estimate of how long it would take by hand comes to about 58 days. This motivates loops.

while

while is a Python keyword followed by a boolean **condition**. The "loopy" solution to the summation problem:

PYTHON

```
total = 0
num = 0
while num < 1_000_000:
    num = num + 1
    total = total + num
print(total)
# Rest of code will follow below this comment
```

The indented lines form the loop body. If the condition is **True**, control enters the body and executes lines sequentially, then loops back to re-check the condition. When the condition becomes **False**, the body is skipped and execution continues after the loop. The body must be indented to separate it from surrounding code. Note that the underscore in **1_000_000** improves readability for large numbers.

Two further problems are described:

1. Read integers until a negative is entered; print the sum of positives (or 0 if none).
2. Read integers until a negative is entered; print the maximum positive (or 0 if none).

Solution to the maximum problem:

PYTHON

```
# Initialize
num = int(input())
max_num = 0
# Loop
while num >= 0:
    if num > max_num:
        max_num = num
    num = int(input())
# Print output
print(max_num)
```

The indented lines make up the loop body; the comments help explain the code.

Loop Control Statements

`break` and `continue` are keywords tied to loops. `break` exits a loop entirely, skipping any code below it. This example finds the LCM of 2, 3, 4:

PYTHON

```
num = 1
while True:
    if (num % 2 == 0) and (num % 3 == 0) and (num % 4 == 0):
        break
    num = num + 1
print(num)
```

`continue` skips to the next iteration, bypassing the remaining code in the current pass. This example prints multiples of 3 up to 50:

PYTHON

```
x = 0
while x < 50:
    x = x + 1
    if x % 3 != 0:
        continue
    print(x)
```

Both statements skip the code that follows them within the loop, but "break exits the loop whereas continue moves to the next iteration." Both examples could be rewritten without `break` or `continue`, which is left as an exercise.

Lesson 3.2 — The for Loop, range() and Iterating Strings

for loop

The `for` loop is an alternative to a while loop. Consider printing the first 5 non-negative integers:

PYTHON

```
for i in range(5):
    print(i) # A dummy line
```

OUTPUT

```
0
1
2
3
4
```

Both `for` and `in` are Python keywords, while `range` is an object representing a sequence of numbers. The second line forms the body of the loop. The loop behaves as follows:

- Each iteration picks up an element from the sequence and prints it to the console.
- Reading left to right, the leftmost element is picked first.
- Processing goes left to right through the sequence.
- After the rightmost element prints, control returns to the first line one final time; with no elements left, control exits the loop.

As with `while` loops and `if - else` blocks, the body of a `for` loop must be indented.

range()

The `range()` function returns a sequence of numbers. `range(5)` produces the sequence 0, 1, 2, 3, 4. More generally, `range(n)` creates the sequence 0, 1, ..., $n - 1$. The following prints all two-digit numbers greater than zero:

PYTHON

```
for i in range(10, 100):
    print(i)
```

Here `range(10, 100)` represents 10, 11, ..., 99. In general, `range(start, stop)` gives `start, start + 1, ..., stop - 1`. The next example prints all even two-digit natural numbers:

PYTHON

```
for i in range(10, 100, 2):
    print(i)
```

`range(10, 100, 2)` represents 10, 12, ..., 98. In general, `range(start, stop, step)` gives `start, start + step, start + 2 * step, ..., last`, where `last` is the largest element less than `stop`. This holds when `step` is positive.

The following three forms are equivalent:

- `range(n)`
- `range(0, n)`
- `range(0, n, 1)`

Using a negative step size, you can create decreasing sequences. This prints all two-digit even numbers greater than zero in descending order:

PYTHON

```
for i in range(98, 9, -2):
    print(i)
```

For a negative `step`, the sequence runs `start, start + step, ...`, with `last` being the smallest element greater than `stop`.

Next, consider:

PYTHON

```
for i in range(5, 5):
    print(i)
```

`range(5, 5)` is an empty sequence, so nothing prints. Another empty-sequence example:

PYTHON

```
for i in range(10, 5):
    print(i)
```

Importantly, neither snippet produces an error. Finally, run and observe the following (flagged as incorrect):

PYTHON

```
##### Alarm! Wrong code snippet! #####
for i in range(0.0, 10.0):
    print(i)
##### Alarm! Wrong code snippet! #####
```

Iterating through Strings

Because a string is a sequence of characters, a `for` loop can iterate through it. This prints each character of the string on its own line:

PYTHON

```
word = 'good'
for char in word:
    print(char)
```

OUTPUT

```
g
o
o
d
```

The output can be enriched with additional code:

PYTHON

```
word = 'good'
count = 1
for char in word:
    print(char, 'occurs at position', count, 'in the string', word)
    count = count + 1
```

OUTPUT

```
g occurs at position 1 in the string good
o occurs at position 2 in the string good
o occurs at position 3 in the string good
d occurs at position 4 in the string good
```

Lesson 3.3 — Nested Loops; while vs for; print end and sep

Nested loops

Here is a problem to consider: count the ordered pairs of positive integers that multiply to 100, where order matters so that (2, 50) and (50, 2) are treated as distinct.

PYTHON

```
count = 0
for a in range(1, 101):
    for b in range(1, 101):
        if a * b == 100:
            count = count + 1
print(count)
```

This is a nested loop. The outer loop steps through values of `a`, and the inner loop steps through values of `b`, with multiple indentation levels. This problem "could have been solved without using a nested loop" and the nested approach is not efficient — a better solution is left as an exercise.

Next problem: count the primes below a positive integer n .

PYTHON

```
n = int(input())
count = 0
for i in range(2, n + 1):
    flag = True
    for j in range(2, i):
        if i % j == 0:
            flag = False
            break
    if flag:
        count = count + 1
print(count)
```

The idea:

- The outer loop steps through 2, 3, ..., n using loop variable `i`.
- Initially assume `i` is prime by setting `flag` to `True`.
- The inner loop tests potential divisors 2, 3, ..., $i-1$ using `j`; note the inner sequence depends on the outer variable `i`.
- If `j` divides `i`, then `i` is not prime, so `flag` becomes `False` and we break out of the inner loop.
- If no `j` divides `i`, the assumption holds and `flag` stays `True`.

- After the inner loop, if `flag` is `True`, we increment `count` for the prime found.

Important points about nesting:

- Nesting is not limited to `for` loops. Valid combinations include: `for` inside `for`, `for` inside `while`, `while` inside `while`, and `while` inside `for`.
- Multiple levels of nesting are allowed.

while versus for

Generally, `for` loops suit cases where the iteration count can be quantified, while `while` loops fit cases where it cannot be known exactly beforehand. That does not mean a `for` loop's iteration count is always constant, as this example shows:

PYTHON

```
n = int(input())
for i in range(n):
    print(i ** 2)
```

The iteration count varies with each different input, but once the input is known, the count is fixed. By contrast:

PYTHON

```
x = int(input())
while x > 0:
    x = int(input())
```

Here the iteration count is only knowable after termination; there is no way to express it as an explicit function of the user input.

print: end, sep

end

Problem: accept a positive integer `n` and print numbers 1 to `n` on one line separated by commas. For `n = 9`, the desired output is:

OUTPUT

```
1,2,3,4,5,6,7,8,9
```

This attempt will not work:

PYTHON

```
n = int(input())
for i in range(1, n + 1):
    print(i, ',')
```

For `n = 9`, it produces:

OUTPUT

```
1 ,  
2 ,  
3 ,  
4 ,  
5 ,  
6 ,  
7 ,  
8 ,  
9 ,
```

The `print` function offers a fix:

PYTHON

```
n = int(input())  
for i in range(1, n):  
    print(i, end = ',')  
print(n)
```

For `n = 9`, this gives the required output:

OUTPUT

```
1,2,3,4,5,6,7,8,9
```

By default, `print()` outputs the expression and then a newline. That behavior is controlled by the `end` argument, whose default is the newline character. Setting `end` to a comma forces `print()` to append a comma rather than a newline. It is named `end` because it is added at the end. Consider this code:

PYTHON

```
print()  
print(end = ',')  
print(1)  
print(1, end = ',')  
print(2, end = ',')  
print(3, end = ',')
```

OUTPUT

```
,1  
1,2,3,
```

The first output line is a newline because the default `end` is `'\n'`, even though nothing is passed. The second `print` passes no expression but sets `end` to `,`, so only a comma appears.

sep

When multiple expressions are passed to `print()`, they appear on the same line separated by a space by default:

PYTHON

```
print('this', 'is', 'cool')
```

OUTPUT

```
this is cool
```

To change or remove that separator, use `sep` :

PYTHON

```
print('this', 'is', 'cool', sep = ',')
```

OUTPUT

```
this,is,cool
```

An empty string works as a separator too:

PYTHON

```
print('this', 'is', 'cool', sep = '')
```

OUTPUT

```
thisiscool
```

end and sep

A final example uses both `end` and `sep` . Accept a positive integer `n` (a multiple of 3) and print this pattern:

OUTPUT

```
|1,2,3|4,5,6|7,8,9|...|n - 2,n - 1,n|
```

For `n = 9` , print:

OUTPUT

```
|1,2,3|4,5,6|7,8,9|
```

Solution:

PYTHON

```
n = int(input())
print('|', end = '')
for i in range(1, n + 1, 3):
    print(i, i + 1, i + 2, sep = ', ', end = '|')
print()
```

The `for` loop steps by 3 starting at 1. Setting `sep` to `,` prints the comma-separated triplet `i, i + 1, i + 2` , while setting `end` to `|` prints the bar after each triplet. The second line ensures a leading `|` . The final `print()` outside the loop moves the console

prompt to the next line after the pattern finishes — you can remove it to observe the difference.

Lesson 3.4 — Formatted Printing: f-strings, format(), Specifiers

Formatted printing

Consider a program that reads a name and prints a greeting:

PYTHON

```
name = input()
print('Hi,', name, '!')
```

With `Sachin` as input, the output is:

OUTPUT

```
Hi, Sachin !
```

The problem: there is an unwanted space after the name — a formatting issue that Python's tools can solve.

f-strings

The first method is the **formatted string literal**, or f-string. The corrected version:

PYTHON

```
name = input()
print(f'Hi, {name}!')
```

OUTPUT

```
Hi, Sachin!
```

The key string here is the f-string `f'Hi, {name}'`. The `f` prefix distinguishes it from a normal string, and when evaluated it produces a string with the variable's value inserted where `{name}` appears. Two elements are essential: the `f` prefix and the curly braces around the variable.

Omitting either one produces different behavior:

PYTHON

```
name = 'Sachin'
print('Hi, {name}!')
print(f'Hi, name!')
```

OUTPUT

```
Hi, {name}!
Hi, name!
```


A rectangle example demonstrates using expressions inside braces:

PYTHON

```
l, b = int(input()), int(input())
print(f'The length of the rectangle is {l} units')
print(f'The breadth of the rectangle is {b} units')
print(f'The area of the rectangle is {l * b} square units')
```

For `l = 4`, `b = 5`, the output is:

OUTPUT

```
The length of the rectangle is 4 units
The breadth of the rectangle is 5 units
The area of the rectangle is 20 square units
```

Note that the last line contains an expression (`l * b`) rather than just a variable. Any valid Python expression is allowed inside the braces, and the interpreter substitutes its value.

A multiplication table example:

PYTHON

```
x = int(input())
print(f'Multiplication table for {x}')
for i in range(1, 11):
    print(f'{x} X {i} \t=\t {x * i}')
```

For input `3`, the output is:

OUTPUT

```
Multiplication table for 3
3 X 1  =   3
3 X 2  =   6
3 X 3  =   9
3 X 4  =  12
3 X 5  =  15
3 X 6  =  18
3 X 7  =  21
3 X 8  =  24
3 X 9  =  27
3 X 10 =  30
```

The `\t` is a tab character placed before and after the `=`. Try removing both tabs and re-running to observe the change.

f-strings can also define string variables, not just be passed to `print()`:

PYTHON

```

name = input()
qual = input()
gender = input()
if qual == 'phd':
    name_respect = f'Dr. {name}'
elif gender == 'male':
    name_respect = f'Mr. {name}'
elif gender == 'female':
    name_respect = f'Ms. {name}'
print(f'Hello, {name_respect}')

```

Guess what this code does.

format()

Another approach uses the string method `format()` :

PYTHON

```

name = input()
print('Hi, {}'.format(name))

```

The curly braces get replaced by the value of `name`. The rectangle example again:

PYTHON

```

l, b = int(input()), int(input())
print('The length of the rectangle is {} units'.format(l))
print('The breadth of the rectangle is {} units'.format(b))
print('The area of the rectangle is {} square units'.format(l * b))

```

The multiplication table with `format()` :

PYTHON

```

x = int(input())
for i in range(1, 11):
    print('{} X {} \t=\t {}'.format(x, i, x * i))

```

The output matches the f-string version. The three pairs of braces are filled left-to-right by the three arguments given to `format()`.

An example with implicit mapping:

PYTHON

```

fruit1 = 'apple'
fruit2 = 'banana'
print('{} and {} are fruits'.format(fruit1, fruit2))

```

The mapping can be made explicit using indices:

PYTHON

```

fruit1 = 'apple'
fruit2 = 'banana'
print('{0} and {1} are fruits'.format(fruit1, fruit2))

```

The integers give the argument index (starting from 0, left to right). Changing the argument order changes the output. A third form uses keyword arguments:

PYTHON

```
fruit1 = 'apple'
fruit2 = 'banana'
print('{string1} and {string2} are fruits'.format(string1 = fruit1, string2 = fruit2))
```

This relies on keyword arguments, which are covered later in the functions chapter and set aside for now.

Format specifiers

Consider:

PYTHON

```
pi_approx = 22 / 7
print(f'The value of pi is approximately {pi_approx}')
```

OUTPUT

```
The value of pi is approximately 3.142857142857143
```

There are too many digits after the decimal. Format specifiers solve this:

PYTHON

```
pi_approx = 22 / 7
print(f'The value of pi is approximately {pi_approx:.2f}')
```

OUTPUT

```
The value of pi is approximately 3.14
```

Inside `{pi_approx:.2f}`, the part before `:` is the variable and the part after is the format specifier `.2f`:

- `.` signifies the decimal point.
- `2` (after the decimal) means exactly two digits after the decimal point, i.e., the value is rounded to two decimal places.
- `f` signifies a `float` value.

A variant with `.3f`:

PYTHON

```
pi_approx = 22 / 7
print(f'The value of pi is approximately {pi_approx:.3f}')
```

OUTPUT

```
The value of pi is approximately 3.143
```

Next, printing three students' marks:

PYTHON

```
roll_1, marks_1 = 'BSC1001', 90.5
roll_2, marks_2 = 'BSC1002', 100
roll_3, marks_3 = 'BSC1003', 90.15
print(f'{roll_1}: {marks_1}')
print(f'{roll_2}: {marks_2}')
print(f'{roll_3}: {marks_3}')
```

OUTPUT

```
BSC1001: 90.5
BSC1002: 100
BSC1003: 90.15
```

To right-align the marks with uniform representation:

PYTHON

```
roll_1, marks_1 = 'BSC1001', 90.5
roll_2, marks_2 = 'BSC1002', 100
roll_3, marks_3 = 'BSC1003', 90.15
print(f'{roll_1}: {marks_1:10.2f}')
print(f'{roll_2}: {marks_2:10.2f}')
print(f'{roll_3}: {marks_3:10.2f}')
```

OUTPUT

```
BSC1001:      90.50
BSC1002:     100.00
BSC1003:      90.15
```

In `{marks_1:10.2f}`, `.2f` rounds the float to two decimals, and the `10` sets the minimum column width. If the number has fewer than 10 characters (including the decimal point), spaces are added before the number to compensate.

Finally, printing integers with formatting via `format()`:

PYTHON

```
print('{0:5d}'.format(1))
print('{0:5d}'.format(11))
print('{0:5d}'.format(111))
print('{:5d}'.format(1111))
print('{:5d}'.format(11111))
print('{:5d}'.format(111111))
```

OUTPUT

```
  1
 11
111
1111
11111
111111
```

Points to note:

- The `d` stands for integer.
- The first three statements include the argument index (`0`) before the `:` ; the last three have nothing before the `:` . Both forms are valid.
- The `5d` sets a minimum column width of 5.
- The first four lines have leading spaces because those integers have fewer than five characters.

Lesson 3.5 — System Libraries: math and random

This lesson covers two more libraries — `math` and `random` — using them to solve interesting problems in mathematics.

math

Consider a sequence of nested square roots: $\sqrt{2}$, $\sqrt{2+\sqrt{2}}$, $\sqrt{2+\sqrt{2+\sqrt{2}}}$, and so on. This sequence converges toward a specific value called its **limit**. Can we estimate this value using what we've learned?

PYTHON

```
import math
x = 0
for n in range(1, 6):
    x = math.sqrt(2 + x)
    print(f'n = {n}, x_n = {x:.3f}')
```

OUTPUT

```
n = 1, x_n = 1.414
n = 2, x_n = 1.848
n = 3, x_n = 1.962
n = 4, x_n = 1.990
n = 5, x_n = 1.998
```

The `sqrt()` function returns the square root of its argument. The results as a table:

N	X_N	APPROXIMATE VALUE
1	$\sqrt{2}$	1.414
2	$\sqrt{2+\sqrt{2}}$	1.848
3	$\sqrt{2+\sqrt{2+\sqrt{2}}}$	1.962
4	$\sqrt{2+\sqrt{2+\sqrt{2+\sqrt{2}}}}$	1.990
5	$\sqrt{2+\sqrt{2+\sqrt{2+\sqrt{2+\sqrt{2}}}}}$	1.998

The sequence appears to approach the value 2. Running the loop for more iterations:

PYTHON

```
import math
x = 0
for n in range(1, 20):
    x = math.sqrt(2 + x)
print(x)
```

After 20 iterations the value is very close to two: `1.99999999999910236`. Since deciding when to stop by trial and error is not ideal, a better approach is to define a **tolerance** — terminate when the difference between successive values falls below a predefined value.

PYTHON

```
import math
x_prev, x_curr = 0, math.sqrt(2)
tol, count = 0.00001, 0
while abs(x_curr - x_prev) >= tol:
    x_prev = x_curr
    x_curr = math.sqrt(2 + x_prev)
    count += 1
print(f'Value of x at {tol} tolerance is {x_curr}')
print(f'It took {count} iterations')
```

random

How can we toss a coin in Python?

PYTHON

```
import random
print(random.choice('HT'))
```

`random` is a library and `choice()` is a function within it that accepts any sequence and returns a randomly chosen element. Here the input is a string, which is a sequence of characters.

The theoretical probability of heads on a coin toss is 0.5. To verify this computationally, we set up an experiment: toss a coin n times, count the heads, and divide by n to get the empirical probability. As n grows large, this should approach 0.5.

PYTHON

```
import random
n = int(input())
heads = 0
for i in range(n):
    toss = random.choice('HT')
    if toss == 'H':
        heads += 1
print(f'P(H) = {heads / n}')
```

Running this for different values of n :

N	$P(H)$
10	0.2
100	0.52
1,000	0.517
10,000	0.5033
100,000	0.49926
1,000,000	0.499983

As expected, the value approaches 0.5 . `random` is quite versatile.

EXERCISE

Rolling a die: `randint(a, b)` returns a random integer N where $a \leq N \leq b$.

PYTHON

```
import random
print(random.randint(1, 6))
```

Find the empirical probability for each face of a die using this function.

Lesson 3.6 — Mathematics and Programming

This lesson explores where mathematics and programming meet, offered as a closing topic for the chapter.

Limits

The lesson begins with the number $\sqrt{2} - 1$. Since it is known that $1 < \sqrt{2} < 2$, it follows that $0 < \sqrt{2} - 1 < 1$. Consider the sequence:

$$a_n = (\sqrt{2} - 1)^n$$

As n grows large, the terms shrink toward zero, because repeatedly multiplying a fraction by itself makes it progressively smaller. Mathematically, the limit as n approaches infinity is zero. This is verified with the following program:

PYTHON

```
import math
n = int(input())          # sequence length
CONST = math.pow(2, 0.5) - 1 # basic term in the sequence
a_n = 1                   # zeroth term
for i in range(n):
    a_n = a_n * CONST      # computing the nth term
print(a_n)
```

Try several values of n . For $n = 100$, the result is 5.27×10^{-39} — so tiny that it may as well be zero for practical purposes.

Recurrence relation

Next comes another fact: for every number n , there exist unique integers x and y such that:

$$(\sqrt{2} - 1)^n = x + y \cdot \sqrt{2}$$

For $n = 1$, this is clear: $x = -1$, $y = 1$. For larger values, the claim can be established by mathematical induction. A sketch of the proof: if $(\sqrt{2} - 1)^n = x_n + y_n \cdot \sqrt{2}$, then

$$(\sqrt{2} - 1)^{n+1} = (x_n + y_n \cdot \sqrt{2}) \cdot (\sqrt{2} - 1) = (2y_n - x_n) + (x_n - y_n) \cdot \sqrt{2} = x_{n+1} + y_{n+1} \cdot \sqrt{2}$$

This equation defines a *recurrence relation* — one where "each new term in the sequence is a function of the preceding terms." With starting values $x_1 = -1$ and $y_1 = 1$, for $n > 0$ the recurrence is:

$$x_{n+1} = 2y_n - x_n \tag{1}$$

$$y_{n+1} = x_n - y_n \tag{2}$$

Loops are handy tools for computing such sequence terms:

PYTHON

```
n = int(input())    # sequence length
x_n, y_n = -1, 1    # x_1 and y_1
for i in range(n - 1):
    x_n, y_n = 2 * y_n - x_n, x_n - y_n
```

Rational Approximation

This recurrence provides a way to approximate $\sqrt{2}$ using rational numbers:

$$\sqrt{2} \approx \frac{-x_n}{y_n}$$

As n grows, the approximation gets increasingly accurate. After 100 iterations, the result is accurate to several decimal places:

$$\frac{228725309250740208744750893347264645481}{161733217200188571081311986634082331709}$$

The lesson closes with a reflection on usefulness: "We don't do things because they are useful. We do them because they are interesting." The point being that interesting things tend to eventually find a use.

4

Functions

Lesson 4.1 — Functions: Introduction and Examples

Introduction

In math, a function takes inputs and produces outputs — for example, $f(x) = x^2$ squares a number. Python functions serve a similar role but are far richer. Here is the mathematical function converted to Python:

PYTHON

```
def f(x):  
    y = x ** 2  
    return y
```

This is the **definition** of function `f`. The `def` keyword defines functions, `f` is the name, and `x` is a parameter. The indented lines form the body — a set of statements describing what the function does. The last line returns the value in `y` using the `return` keyword.

Running this produces no output, because functions only run when called. A **function call** looks like this:

PYTHON

```
def square(x):  
    y = x ** 2  
    return y  
print(square(2))
```

OUTPUT

4

Here `square(2)` is a function call. The `x` in the definition is the **parameter**; the value passed in the call (here, 2) is the **argument**. This convention holds throughout the lesson.

A mental model for understanding functions:

- Parameters are the function's inputs.
- The body is the sequence of steps transforming input into output.
- The return statement communicates the output to the rest of the code.

Examples

The following examples focus on syntactical aspects of function definitions.

Functions could have multiple parameters:

PYTHON

```
# This function computes the area of a rectangle.  
# Length and breadth are the parameters  
def area(l, b):  
    return l * b
```

Functions could have no parameters:

PYTHON

```
def foo():  
    return "I don't like arguments visiting me!"
```

Functions could have no return value:

PYTHON

```
def foo():  
    print("I don't like talking to the outside world!")  
foo()
```

OUTPUT

```
I don't like talking to the outside world!
```

Note that you did not need `print(foo())` — just calling `foo()` works since the print statement is already inside. But what if you type `print(foo())` ?

OUTPUT

```
I don't like talking to the outside world!  
None
```

If no explicit return statement is present, `None` is the default return value. The interpreter first evaluates `foo()` (producing the first output line), and since it has no explicit return, it returns `None` (the second output line).

A minimal Python function looks like:

PYTHON

```
def foo():  
    pass
```

`pass` is a keyword; when the interpreter reaches it, it performs no computation and moves on. This is minimal because it has only the essentials for a syntactically valid definition: a name and at least one body statement. Such functions have their place. While writing complex code, a coder may realize they need a function for a task but not know the implementation details. They add a minimal named function as a to-do item, implementing it when the need arises.

Functions could have multiple return statements, but the first executed return exits the function:

PYTHON

```
def foo():
    return 1
    return 2
```

`foo()` always returns 1; the last line is redundant. A non-redundant example:

PYTHON

```
def evenOrOdd(n):
    if n % 2 == 0:
        return 'even'
    else:
        return 'odd'
print(evenOrOdd(10))
print(evenOrOdd(11))
```

OUTPUT

```
even
odd
```

With an even argument, the first return runs; with an odd argument, the else return runs.

Functions could return multiple values:

PYTHON

```
# Accept only positive floating point numbers
def bound(x):
    lower = int(x)
    upper = lower + 1
    return lower, upper
y = 7.3
l, u = bound(y)
print(f'{l} < {y} < {u}')
```

The exact mechanism will become clear in the lesson on tuples. Here the first returned value goes into `l` and the second into `u`.

Functions must be defined before being called; the call cannot precede the definition:

PYTHON

```
##### Alarm! Wrong code snippet! #####
print(f(5))
def f(x):
    return x ** 2
##### Alarm! Wrong code snippet! #####
```

This throws a `NameError`. The interpreter executes top to bottom; at the call line, `f` is a name never seen before, so it raises a `NameError` — which occurs when referencing a name the interpreter has not seen.

Function calls could be used in expressions:

PYTHON

```
def square(a):
    return a ** 2

x, y, z = int(input()), int(input()), int(input())
if square(x) + square(y) == square(z):
    print(f'{x}, {y} and {z} form the sides of a right triangle with {z} as the hypotenuse')
```

Function calls cannot be assigned values:

PYTHON

```
##### Alarm! Wrong code snippet! #####
def foo():
    return True

foo() = 1
##### Alarm! Wrong code snippet! #####
```

This throws a `SyntaxError`.

Functions can be called from within other functions:

PYTHON

```
def foo():
    print('I am inside foo')
def bar():
    print('I am inside bar')
    print('I am going to call foo')
    foo()
print('I am outside both foo and bar')
bar()
print('I am outside both foo and bar')
```

Functions can be defined inside other functions:

PYTHON

```
def foo():
    def bar():
        print('bar is inside foo')
    bar()
foo()
```

Try calling `bar()` outside `foo()`. What do you observe?

Docstrings

Consider this function:

PYTHON

```
def square(x):
    """Return the square of x."""
    return x ** 2
```

The string just below the function definition is a docstring. From the Python docs:

"A docstring is a string literal that occurs as the first statement in a module, function, class, or method definition."

Such a docstring becomes the `__doc__` special attribute of that object. Ignore unfamiliar terms like "module" and "class" for now and focus on functions. Adding docstrings is good practice. It may not be needed for simple, obvious functions, but as function complexity grows, docstrings can be a lifesaver for other programmers reading your code.

The docstring can be accessed via the `__doc__` attribute:

PYTHON

```
print(square.__doc__)
```

This gives `Return the square of x.` as output.

Lesson 4.2 — Arguments and Call by Value

Arguments

Python provides several ways for passing arguments to functions.

Positional arguments

Every function shown so far has relied on positional arguments, where an argument's position in the call determines which parameter receives it. Consider this problem: write a function taking three positive integers `x`, `y`, and `z`, returning `True` when they form a right triangle with legs `x` and `y` and hypotenuse `z`, else `False`.

PYTHON

```
def isRight(x, y, z):
    if x ** 2 + y ** 2 == z ** 2:
        return True
    return False

print(isRight(3, 4, 5)) # 3 is passed to x, 4 is passed to y, 5 is passed to z
print(isRight(5, 4, 3)) # 5 is passed to x, 4 is passed to y, 3 is passed to z
```

OUTPUT

```
True
False
```

Arguments bind to parameters based on their position in the call. Positional arguments are also known as required arguments — they cannot be omitted, and supplying too many raises an error. There must be exactly as many arguments as parameters. Run this and note the error:

PYTHON

```
##### Alarm! Wrong code snippet!
isRight(3, 4)
isRight(3, 4, 5, 6)
##### Alarm! Wrong code snippet!
```

Keyword arguments

Keyword arguments add flexibility. Reusing the same problem with modified calls:

PYTHON

```
# The following is just a function call.  
# We are not printing anything here.  
isRight(x = 3, y = 4, z = 5)
```

Here the parameter names are stated explicitly and values assigned with `=`. A benefit is that it lowers the chance of ordering arguments incorrectly:

PYTHON

```
isRight(3, 4, 5)    # intended call  
isRight(5, 4, 3)    # actual call  
isRight(x = 3, y = 4, z = 5) # same as intended call  
isRight(z = 5, y = 4, x = 3) # same as intended call
```

Keyword and positional arguments can be mixed in one call:

PYTHON

```
isRight(3, y = 4, z = 5)
```

Now try this:

PYTHON

```
#### Alarm! Wrong code snippet! ####  
isRight(x = 3, 4, 5)  
#### Alarm! Wrong code snippet! ####
```

The interpreter raises a `TypeError` reading `positional argument follows keyword arguments`. The positional arguments `4` and `5` appear after the keyword argument `x = 3`. Whenever both types are present, keyword arguments must come last — sensible, since positional arguments depend heavily on position and belong at the front.

What about this call?

PYTHON

```
#### Alarm! Wrong code snippet! ####  
isRight(3, x = 3, y = 4, z = 5)  
#### Alarm! Wrong code snippet! ####
```

This raises a `TypeError: isRight() got multiple values for argument x`. Each parameter needs exactly one argument — no more, no less — supplied either positionally or as a default, but not both.

Default arguments

Imagine a neighborhood map where grid lines are roads for cars. To go from point *O* to point *P* on foot, the shortest path is straight along line *OP*, the **Euclidean distance**. In a car you must follow the grid, covering *OM* + *MP*, the **Manhattan distance**.

Suppose a self-driving car startup uses both metrics, calling Euclidean distance 10 times and Manhattan 1000 times. Representing them as functions makes sense:

PYTHON

```
# Assume that 0 is the origin
# ALL distances are computed from the origin
def euclidean(x, y):
    return pow(x ** 2 + y ** 2, 0.5)

def manhattan(x, y):
    return abs(x) + abs(y)
```

This code works but ignores that Manhattan is used a hundred times more often. Default arguments help:

PYTHON

```
def distance(x, y, metric = 'manhattan'):
    if metric == 'manhattan':
        return abs(x) + abs(y)
    elif metric == 'euclidean':
        return pow(x ** 2 + y ** 2, 0.5)
```

The `metric` parameter defaults to `'manhattan'`. Calling without passing `metric`:

PYTHON

```
print(distance(3, 4))
```

This gives `7`. With no value provided, the default `'manhattan'` was used. Anywhere the Manhattan distance is needed, `distance(x, y)` can replace it.

Key points to remember:

- Parameters assigned a value in the definition are called default parameters.
- Default parameters always come at the end of the parameter list in a definition.
- The argument for a default parameter is optional in a call.
- Such an argument may be passed positionally or as a keyword argument.

Illustrating these:

PYTHON

```
#### Alarm! Wrong code snippet! ####
def distance(metric = 'manhattan', x, y):
    if metric == 'manhattan':
        return abs(x) + abs(y)
    elif metric == 'euclidean':
        return pow(x ** 2 + y ** 2, 0.5)
#### Alarm! Wrong code snippet! ####
```

This raises a `SyntaxError: non-default argument follows default argument`. The default parameter must come last. Different valid ways of passing arguments with defaults:

PYTHON

```
distance(3, 4)
distance(3, 4, 'manhattan')
distance(3, 4, metric = 'manhattan')
```

All three calls are equivalent: the first uses the default, the second passes `'manhattan'` positionally, and the third passes it as a keyword argument.

Call by value

Consider this code:

PYTHON

```
def double(x):
    x = x * 2
    return x

a = 4
print(f'before function call, a = {a}')
double(a)
print(f'after function call, a = {a}')
```

OUTPUT

```
before function call, a = 4
after function call, a = 4
```

The value of `a` is unaffected by the function. When `double(a)` runs, the value in `a` is assigned to parameter `x` — arguments are passed by assignment in Python, so effectively `x = a` occurs. Passing a variable's value like this is called **call by value**.

Consider the following code:

PYTHON

```
def square(x):
    return x * x

x = 10
x_squared = square(x)
```

Using the same name for the function parameter and the passed argument is poor practice. It is better to differentiate their names to avoid confusion and improve readability. How the inner `x` relates to the outer `x` will be covered in the next lesson on scopes. The code can be rewritten as:

PYTHON

```
def square(num):
    return num * num

x = 10
x_squared = square(x)
```

Lesson 4.3 — Scope, Namespaces, locals and globals

Scope

Consider a function `foo()` that assigns `x = 1` internally and prints messages, then calls `foo()` and tries `print(x)` from outside. This produces a `NameError` because `x` exists only inside the function.

The key idea: the region of code where a name can be referenced is its **scope**. Referencing a variable outside its scope raises a `NameError`.

Local vs Global

When `x` is assigned inside a function, its scope is **local** — it only exists inside that function. The textbook uses a memorable metaphor comparing functions to black holes that do not let variables escape.

By contrast, a variable `y = 10` defined in the main program is **global** and can be referenced from anywhere (including inside functions) after being defined. There is a caveat: if a same-named variable is defined inside the function, behavior changes.

The rules (adapted from the Python FAQ) state that assigning a value to a variable anywhere in a function makes it local, while a variable only *referenced* (never assigned) inside a function is treated as global.

Function parameters are also local — `double(x)` with `print(x)` outside will raise a `NameError`.

Examples

Variant-1 defines `y = 10` after the function definition but before calling `foo()` — this works fine. What matters is that `y` is defined before the call.

Variant-2 defines `y = 10` *after* calling `foo()`, which raises a `NameError` since `y` is not defined yet at call time.

A trickier case uses `x = 10` inside `foo` and `x = 100` outside:

```
OUTPUT
x inside foo = 10
x outside foo = 100
```

These are different variables. The inside `x` is local (assigned in the function); the outside `x` is global and inaccessible from within `foo`.

Namespaces

To understand name resolution, the lesson introduces namespaces. A namespace is a lookup table — specifically a dictionary — mapping names to objects.

`globals()`

Variables defined in the main program live in the `globals` namespace. Calling `print(globals())` shows names like `x`, `avar`, and `foo` mapped to their objects. Functions map to something like `<function foo at 0x7f8ecd2aa1f0>`, where the hex value is the memory location of the definition.

You can also check membership:

PYTHON

```
print('x' in globals())
print('avar' in globals())
print('foo' in globals())
```

All three print `True`.

locals()

A variable `y` assigned inside a function is not in `globals`. `print('y' in globals())` returns `False`. The interpreter creates a separate **local namespace** each time a function is called:

PYTHON

```
def foo():
    y = 2.0
    print('Is y in locals?', 'y' in locals())

foo()
print('Is y in globals?', 'y' in globals())
```

OUTPUT

```
Is y in locals? True
Is y in globals? False
```

Scope and Namespaces

For every function call, a local namespace holds the names and objects defined in that function:

PYTHON

```
def foo():
    print(y)
    print(locals())
    x = 1
    print(locals())

y = 10
foo()
```

OUTPUT

```
10
{}
{'x': 1}
```

Since `y` is only referenced, it stays global; `x` is assigned so it enters the local namespace. When control exits the function, its namespace is deleted.

Name resolution protocol (the interpreter checks in order):

- First the local namespace for that call; if present, use that value.
- Otherwise the global namespace; if present, use that value.
- Otherwise the `built-in` namespace (covered at the end).
- If found nowhere, raise a `NameError`.

Revisiting the earlier example: when `foo()` is called, a local namespace is created; `x` enters it at assignment; the reference uses the local value `10`; after exit, the namespace is deleted; and the final line uses the global `100`.

global keyword

Consider:

PYTHON

```
def foo():
    print(x)
    x = x + 1

x = 10
foo()
```

This raises `UnboundLocalError: local variable 'x' referenced before assignment`. Because `x` is assigned in the function, it is local, so referencing it before assignment fails. The outer `x = 10` does not count since it is global.

To reuse the global `x` inside the function, use the `global` keyword:

PYTHON

```
def foo():
    global x
    print(f'x inside foo = {x}')
    x = x + 1
    print(f'x inside foo = {x}')

x = 10
print(f'x outside foo = {x}')
foo()
```

OUTPUT

```
x outside foo = 10
x inside foo = 10
x inside foo = 11
```

Declaring `x` as global means no new local variable is created despite the assignment.

Built-ins

Built-in functions like `print`, `int`, and `input` are also names, resolved at runtime, defined in a separate `builtins` namespace.

This runs without error (but should not be done):

PYTHON

```
##### Never do something like this! #####
print = 1
##### Never do something like this! #####
```

Syntactically it is valid. However:

PYTHON

```
##### Alarm! Wrong code snippet! #####  
print = 1  
print(1)  
##### Alarm! Wrong code snippet! #####
```

This throws a `TypeError` because the name `print` was reassigned to an `int`.

The built-in namespace is the final stage of name resolution.

WARNING

Using a built-in name as a variable is "a very bad practice that should be avoided at any cost!"

Lesson 4.4 — Recursion and Caution in Recursion

Function calling Function

This lesson explores how functions can call other functions, and introduces recursion. Here is a program demonstrating chained function calls:

PYTHON

```
def first():  
    second()  
    print('first')  
  
def second():  
    third()  
    print('second')  
  
def third():  
    print('third')  
  
first()
```

OUTPUT

```
third  
second  
first
```

The lesson introduces a visualization approach called the **traffic-signal method**. A simple function (one that calls no other functions) can be in one of two states: *ongoing* (control is inside executing a line) or *completed* (all lines executed, control has exited — either via `return` or reaching the end, where `None` is returned by default).

A function that calls another function can be in three states: *ongoing*, *suspended*, or *completed* — color coded like a traffic signal. Since `third()` calls no other function, it never enters the suspended state. The print in `third` runs first, so `third` appears first in the

output. Then control returns to the most recent suspended function (`second`), which prints and completes, then `first` prints and completes.

Recursion

A recursive function calls itself within its own body. The classic example is factorial:

PYTHON

```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n - 1)
```

When the interpreter hits the recursive call, it suspends `fact(n)` and begins executing `fact(n - 1)`. When `fact(0)` is called, the condition is `True`, returning `1` — this is the **base-case**. Without a base-case, recursion never terminates. After the base case, control transfers back through the suspended calls: `fact(1)`, then `fact(2)`, and so on until `fact(4)` returns `24`.

Caution in Recursion

Fibonacci series

The Fibonacci series: 1, 1, 2, 3, 5, 8, ... Each term is the sum of the two preceding terms.

With $x_1 = x_2 = 1$, for all $n > 2$ ($n \in \mathbb{N}$): $x_n = x_{n-1} + x_{n-2}$

PYTHON

```
def fibo(n):
    if n == 1 or n == 2:
        return 1
    return fibo(n - 1) + fibo(n - 2)
```

Calling `fibo(40)` takes a very long time due to wasteful repeated computation. The recursion tree shows how `fibo(3)` and `fibo(1)` are computed twice, and `fibo(2)` thrice. Larger values like `50` cause even more redundant work.

To measure runtime, the `time` library helps:

PYTHON

```
import time

def fibo(n):
    if n == 1 or n == 2:
        return 1
    return fibo(n - 1) + fibo(n - 2)

start = time.time()
fibo(40)
end = time.time()
print(f'It took approximately {round(end - start)} seconds.')
```

This takes almost a minute in a standard Python repl. A more efficient iterative solution:

PYTHON

```
import time

def fibo(n):
    if n == 1 or n == 2:
        return 1
    x_prev, x_curr = 1, 1
    while n > 2:
        x_prev, x_curr = x_curr, x_prev + x_curr
        n -= 1
    return x_curr

start = time.time()
fibo(40)
end = time.time()
print(f'It took approximately {round(end - start)} seconds.')
```

The line `x_prev, x_curr = x_curr, x_prev + x_curr` uses multiple simultaneous assignment: the right-hand side is evaluated first, then those values are simultaneously assigned to the containers on the left. A fuller explanation comes with tuples in the next chapter.

Counting Function Calls

You can count how many times a function is called using a global variable:

PYTHON

```
def fact(n):
    global count
    count = count + 1
    if n == 0:
        return 1
    return n * fact(n - 1)

count = 0
fact(4)
print(count)
```

This is one of the legitimate uses of global variables.

Turtles all the way down

A recursive function without a base case is problematic. The simplest pathological example:

PYTHON

```
##### Alarm! Bad code snippet! #####
def foo():
    foo()
##### Alarm! Bad code snippet! #####
```

Calling `foo()` produces a `RecursionError` with the message `maximum recursion depth exceeded`. The limit is usually 1000 in most systems — more than 1000 recursive calls triggers the error. To check the limit:

PYTHON

```
import sys  
print(sys.getrecursionlimit())
```


5

Lists and Tuples

Lesson 5.1 — Lists: Introduction, Iterating, Growing, Operations

Introduction

A list in Python is a data structure used to store a sequence of objects. Some examples:

PYTHON

```
numbers = [1, 2, 3, 4, 5]
letters = ['a', 'b', 'c', 'd']
words = ['this', 'is', 'a', 'list']
```

Lists can be printed like other types. `print(numbers)` gives this output:

OUTPUT

```
[1, 2, 3, 4, 5]
```

Lists may hold objects of different data types:

PYTHON

```
mixture = [1, 1.0, '1', True]
```

Lists have their own data type — `list` — and you can check whether a variable holds one:

PYTHON

```
numbers = [1, 2, 3]
print(type(numbers))
print(isinstance(numbers, list))
```

The `len` function counts the elements:

PYTHON

```
numbers = [1, 2, 3]
print(f'This list has {len(numbers)} elements in it')
```

Lists support indexing and slicing, which behave just as they did for strings:

PYTHON

```
numbers = [1, 2, 3, 4]
print(numbers[0], numbers[1], numbers[2], numbers[3])
print(numbers[1 : 3])
print(numbers[-2])
```

Iterating through lists

Since a list is a sequence, you can loop through it with `for` — one of the main uses of the `for` loop:

PYTHON

```
# Method-1
numbers = [1, 2, 3, 4]
for num in numbers:
    print(num)
```

Here the loop variable `num` picks one item at a time; the body simply prints it. The same result can be achieved with a `while` loop:

PYTHON

```
# Method-2
numbers = [1, 2, 3, 4]
index = 0
while index < len(numbers):
    print(numbers[index])
    index += 1
```

You can also iterate over the indices using `range`:

PYTHON

```
# Method-3
numbers = [1, 2, 3, 4]
for index in range(len(numbers)):
    print(numbers[index])
```

Since `len(numbers)` equals `4`, the range sequence is `0, 1, 2, 3`, and `index` iterates through it. Methods 2 and 3 are alike — both walk through the indices and use indexing to reach each element — the difference being `while` versus `for`. Method-1 differs by pulling elements directly from the sequence.

Growing a list

Lists are commonly used to gather a collection of items, often starting from an empty list. Python offers two ways to make one:

PYTHON

```
list1 = []
list2 = list()
```

Both are empty. The interpreter ignores spaces between the braces, so `list1 = [ ]` works too. To add items, there are two approaches:

PYTHON

```
list1 = list1 + [1]
print(list1)
list2 = list2.append(1)
print(list2)
```

Both lists end up with one element. The first approach is **list concatenation** — joining two lists together, much like coupling two train compartments, and similar to string

concatenation. The second uses the `append` method (a function defined for the `list` type), which adds elements at the end.

Consider this problem:

Generate the list of positive integers less than 100 that are divisible by 3.

At least two solutions exist. First, with `while` :

PYTHON

```
# Method-1
num = 3
nums_div = []
while num < 100:
    nums_div.append(num)
    num += 3
```

And with `for` :

PYTHON

```
# Method-2
nums_div = []
for num in range(3, 100, 3):
    nums_div.append(num)
```

Operations on Lists

We have seen the `+` operator with lists:

PYTHON

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list12 = list1 + list2
print(list12)
list21 = list2 + list1
print(list21)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
[4, 5, 6, 1, 2, 3]
```

Order matters during concatenation. Next, the `*` operator:

PYTHON

```
list1 = [0] * 5
print(list1)
list2 = [1, 2, 3] * 3
print(list2)
```

OUTPUT

```
[0, 0, 0, 0, 0]
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Two lists are equal when they hold the same sequence of elements:

PYTHON

```
l1 = [1, 2, 3]
l2 = [1, 2, 3]
l3 = [3, 2, 1]
print(l1 == l2)
print(l2 == l3)
```

OUTPUT

```
True
False
```

Lists can also be compared with `>` or `<`, using lexicographic ordering — much like string comparison, which was covered in the first chapter.

LEXICOGRAPHIC ORDERING

First element from both lists are compared. If they differ this determines the outcome of the comparison. If they are equal, then the second element of both the lists are compared. This process continues until either list is exhausted.

Some example comparisons:

PYTHON

```
print([1, 2] < [2, 1])
print([1] < [1, 2, 3])
print([2, 3, 4] < [3])
print([] < [1])
```

All four give `True`.

Useful Functions

Some built-in functions that work on lists:

`sum` adds up the elements of a list of numbers:

PYTHON

```
a = [1, 2, 3]
print(sum(a))
```

`max` and `min` find the maximum and minimum values respectively:

PYTHON

```
a = [1, 2, 3]
print(min(a), max(a))
```

What happens if `a` is a list of strings? What would `max(a)` and `min(a)` produce?

`sorted` returns a sorted list:

PYTHON

```
a = [2, 1, 3]
print(sorted(a))
```

We have met the `range` object and seen its usefulness for iterating over a sequence, so far tied to the `for` loop. Now it steps outside that role:

PYTHON

```
numbers = range(10)
print(numbers)
```

This outputs `range(0, 10)` — a sequence you can iterate over. Python lets you turn it into a list:

PYTHON

```
numbers = list(range(10))
print(numbers)
```

This gives `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.

Lesson 5.2 — Lists: Mutability and Call by Reference

Mutability

Consider a scenario: you work at a company analyzing cricket matches, and a colleague recording IPL runs has a broken "o" key. He uses the letter "O" instead, and you must write a program to replace every "O" with the number 0.

PYTHON

```
runs = [1, 4, 2, 'O', 4, 'O'] # the data for one over is given here
print(runs)
for i in range(len(runs)):
    if runs[i] == 'O':
        runs[i] = 0
print(runs)
```

The key line is `runs[i] = 0`. This updates a list "in-place," which Python allows because lists are mutable — unlike strings, which are immutable. But "reckless exercise of power always results in instability," shown by this example:

PYTHON

```
list1 = [1, 2, 3]
list2 = list1
list2[0] = 100
print(list1)
print(list2)
```

Both produce identical results even though only `list2` was modified:

OUTPUT

```
[100, 2, 3]
[100, 2, 3]
```

To understand this, consider the built-in `id` function. Every Python object has a unique identity, and `id(x)` returns it. The docs describe it as "guaranteed to be unique among simultaneously existing objects." In this implementation, the id is the object's memory address.

The assignment `list2 = list1` does not create a new object but instead creates another name (an alias) for the same object — comparable to a nickname. To check whether two names point to the same object, use the `is` keyword:

PYTHON

```
list1 = [1, 2, 3]
list2 = list1
list2[0] = 100
print(list1 is list2)
```

This prints `True`. Another scenario:

PYTHON

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
print(list1 == list2)
print(list1 is list2)
```

OUTPUT

```
True
False
```

Equality and identity differ: the first line checks equality, the second checks identity. Here the two lists point to different objects (different identities) but hold the same sequence, so they are equal.

To create a genuine copy so that updating one does not affect the other, Python offers three ways:

PYTHON

```
list1 = [1, 2, 3]
list2 = list(list1)
list3 = list1[:]
list4 = list1.copy()

list2[0] = 100
list3[0] = 200
list4[0] = 300

print(list1, list2, list3, list4)
print(list1 is not list2, list1 is not list3, list1 is not list4)
```

OUTPUT

```
[1, 2, 3] [100, 2, 3] [200, 2, 3] [300, 2, 3]
True True True
```

Explanations:

- `list(list1)` : passing `list1` to the `list` function returns a new list with the same elements.
- `list1[:]` : slicing the list produces a new list object; with no `start` or `stop`, they default to `0` and `len(list1)`, so the whole list is returned as a brand new object.
- `list1.copy()` : the `copy` method defined for list objects is used.

The last two lines confirm all three copying methods work.

Call by reference

Mutability affects how lists behave in functions. Two snippets are compared:

PYTHON

```
# Snippet-1
def foo():
    L.append(1)

L = [0]
print(f'L before: {L}')
foo()
print(f'L after: {L}')
```

Snippet-1 has no parameters. Since `L` is never assigned a new value inside `foo`, its scope stays global.

PYTHON

```
# Snippet-2
def foo(L_foo):
    L_foo.append(1)
    print(L is L_foo)

L = [0]
print(f'L before: {L}')
foo(L)
print(f'L after: {L}')
```

Here `L_foo` is a parameter with local scope, yet modifying it changes `L` outside the function because both point to the same object. The function call resembles an assignment `L_foo = L`, making `L_foo` another name bound to the same object. This is termed **call by reference**: whenever a mutable variable is passed to a function, references to the object are passed.

The takeaway: modifying mutable objects inside a function produces side effects outside it. To avoid these, create a new list object:

PYTHON

```
def foo(L_foo):
    L_foo.append(1)
    print(L is L_foo)

L = [0]
print(f'L before: {L}')
foo(list(L))
print(f'L after: {L}')
```

Now `foo` produces no side effects. The call could instead use `foo(L[:])` or `foo(L.copy())`.

Lesson 5.3 — Lists: Simulating an IPL Innings

Simulating an IPL Innings

This lesson revisits the challenge of recording runs scored on every ball of an IPL match. A T20 innings has 20 overs of 6 balls each. Assuming all deliveries are fair with no extras, that leaves exactly 120 numbers to record, each between 0 and 6. A list is a good way to store this data for further processing.

To simulate an innings, the `random` library is used:

PYTHON

```
import random
runs = random.choices([0, 1, 2, 3, 4, 5, 6], k = 120)
print(type(runs))
print(len(runs))
```

The `choices` function samples uniformly (with replacement) from the seven input numbers. In summary, it:

- Picks a number at random from `[0, 1, 2, 3, 4, 5, 6]`, each equally likely.
- Copies it to the output list, leaving the original list undisturbed.
- Repeats this process 120 times.

To verify the counts are roughly equal:

PYTHON

```
for run in [0, 1, 2, 3, 4, 5, 6]:
    print('{} appears {} times'.format(run, runs.count(run)))
```

Here, `runs.count(run)` returns how many times `run` appears in the list; `count` is a list method.

OUTPUT

```
0 appears 19 times
1 appears 20 times
2 appears 19 times
3 appears 16 times
4 appears 18 times
5 appears 11 times
6 appears 17 times
```

The counts are close, but not realistic: 5 runs are rarely seen in cricket, and 0, 1, 2 are far more common than 3, 4, 6. Preferences can be expressed using a `weights` keyword argument:

PYTHON

```
import random
# choices is distributed over multiple lines
# this is done to improve readability
runs = random.choices([0, 1, 2, 3, 4, 5, 6],
                      weights = [30, 30, 20, 5, 10, 0, 5],
                      k = 120)
for run in [0, 1, 2, 3, 4, 5, 6]:
    print('{} appears {} times'.format(run, runs.count(run)))
print(f'Total number of runs scored = {sum(runs)}')
```

OUTPUT

```
0 appears 32 times
1 appears 34 times
2 appears 32 times
3 appears 7 times
4 appears 12 times
5 appears 0 times
6 appears 3 times
Total number of runs scored = 185
```

Here `sum(runs)` (a built-in function) totals the list elements. The `weights` argument can be understood with this table:

RUN	WEIGHT
0	30
1	30
2	20
3	5
4	10
5	0
6	5
Total	100

The weight reflects the importance given to a run. From the table, 0 and 1 each occur 30% of the time, 6 occurs 5% of the time, and so on. The `choices` function respects this distribution while selecting items.

To find when the first six was scored:

PYTHON

```
first_six_ball = runs.index(6) + 1
print(first_six_ball)
```

The `index` method takes an element and returns its first occurrence in the list. Since the ball number is one more than the index, `1` is added. What if the input is not present in the list?

PYTHON

```
first_five_ball = runs.index(5)
print(first_five_ball)
```

Since `5` never occurs, this raises a `ValueError` with the message `5 is not in list`. Care is needed when using `index`. The same task could be done differently:

PYTHON

```
for ball, run in enumerate(runs):
    if run == 6:
        print(f'The first six was hit at ball number {ball + 1}')
        break
```

The `enumerate` object is useful for accessing both an element and its index while iterating. It yields pairs of `(index, list[index])`, effectively giving two loop variables — the index and the element.

To find how many balls it took to score the last 50 runs, reversing the list makes iteration easier:

PYTHON

```
balls = 0
last_runs = 0
for run in reversed(runs):
    last_runs += run
    balls += 1
    if last_runs >= 50:
        print(f'It took {balls} balls to score the last 50 runs.')
        break
```

The `reversed` object lets you iterate in reverse order without changing the original list. Finally, to check whether the batsmen ran three runs at any point (regardless of when):

PYTHON

```
three_existence = 3 in runs
print(three_existence)
```

Just as the `in` keyword was earlier used to check for one string inside another, here it checks list membership. This prints `True` if 3 is an element in `runs` and `False` otherwise.

Lesson 5.4 — List Methods; Stack and Queue; split and join

List Methods

insert

So far we have covered list methods such as `append`, `count`, and `index`. There are additional useful methods to explore. The `insert` method places an element at a specified position in a list:

PYTHON

```
L = [1, 1, 2, 3, 8]
L.insert(4, 5)
print(L)
```

The signature `list.insert(index, object)` puts `object` before `index` in the list. Here, `5` is inserted before index `4`. A few more examples:

PYTHON

```
L = [10, 20, 30]
L.insert(0, 5)      # L becomes [5, 10, 20, 30]
L.insert(2, 15)     # L becomes [5, 10, 15, 20, 30]
L.insert(4, 25)     # L becomes [5, 10, 15, 20, 25, 30]
L.insert(len(L), 35) # L becomes [5, 10, 15, 20, 25, 30, 35]
L.insert(20, 40)    # L becomes [5, 10, 15, 20, 25, 30, 35, 40]
```

When the index exceeds the list's length, the element is appended to the end. `insert` shines when adding to the front of a list, whereas `append` handles additions at the end.

pop

Consider this code:

PYTHON

```
L = ['a', 'b', 'c', 'd', 'e', 'f']
index = 1
x = L.pop(index)
print(f'The element {x} at index {index} was removed from the list')
print(f'The current list is {L}')
```

`L.pop(index)` deletes and returns the element at `index`. When no argument is given, `index` defaults to `-1`, making it a default argument for `pop`. A value of `-1` removes the last element:

PYTHON

```
L = ['a', 'b', 'c', 'd', 'e', 'f']
x = L.pop()
print(f'The current list is {L}')
```

What happens if you supply an index that is out of range?

reverse

You can reverse a list in-place:

PYTHON

```
L = [1, 2, 3, 4, 5]
print('Before:', L, id(L))
L.reverse()
print('After:', L, id(L))
```

It is "in-place" because the list keeps the same `id` before and after — it is the same object. Care is needed with in-place operations. A frequent mistake looks like:

PYTHON

```
L = [1, 2, 3, 4, 5]
L = L.reverse()
print(L)
```

This prints `None`, since `reverse` returns nothing. If you want both the original and its reversed version, do this:

PYTHON

```
L = [1, 2, 3, 4, 5]
L_reversed = L.copy()
L_reversed.reverse()
print('Original list:', L)
print('Reversed list:', L_reversed)
```

Why was the copy needed?

sort

Another handy in-place method is `sort`:

PYTHON

```
L = [2, 1, 5, 6, 4, 3]
print('Before', L)
L.sort()
print('After', L)
```

Although the call looks simple, sorting is a non-trivial algorithm. Sorting algorithms will be covered in the next course on data structures and algorithms.

remove

Now for some destructive functions:

PYTHON

```
L = [1, 2, 3, 4, 5] * 2
print('Before', L)
L.remove(1)
print('After', L)
```

`L.remove(x)` deletes the leftmost occurrence of `x`. Attempting to remove a missing element raises a `ValueError` reading `list.remove(x): x not in list`. A safe removal approach:

PYTHON

```
# x is the item to be removed; L is the list
if x in L:
    L.remove(x)
```

How does `remove` differ from `pop`?

Stack

Combining a list with `append` and `pop` mimics a data structure called a **stack**, where the last item added is the first removed — like a pile of books, where the topmost (most recent) book comes off first. The mnemonic is LIFO: Last In First Out.

PYTHON

```
# Start with an empty stack
stack = [ ]
# Append items to end of the stack; also called a push operation
stack.append('Harry Potter and the Philosopher\'s Stone')
stack.append('Harry Potter and the Chamber of Secrets')
# State of the stack
print(stack)
# Remove items from the end of the stack; also called a pop operation
stack.pop()
# State of the stack
print(stack)
```

Queue

A list paired with `insert` and `pop` simulates a **queue**, where the first item added is the first removed — like a real-life line at a billing counter, where the first person served is the first to leave. The mnemonic is FIFO: First In First Out.

PYTHON

```
# Start with an empty queue
queue = [ ]
# Insert elements at the beginning of the queue
queue.insert(0, 'Customer-1')
queue.insert(0, 'Customer-2')
# State of the queue
print(queue)
# Remove items from the queue
queue.pop()
# State of the queue
print(queue)
```

Strings and Lists

split

Lists come up often when processing strings. Consider this problem:

Accept a sentence as input and find the number of words in it. Assume that it is a simple sentence with a single space separating consecutive words. There are no other punctuation marks in the sentence.

First, a solution without lists:

PYTHON

```
sentence = 'this sentence is false' # a simple sentence
count = 1
for char in sentence:
    if char == ' ':
        count += 1
print(count)
```

We scanned character by character, counting spaces; the word count is one more than the number of spaces. As an aside, this sentence is a paradoxical statement — it cannot be true or false, since assuming either leads to the other. Now the list-based approach:

PYTHON

```
sentence = 'this sentence is false' # a simple sentence
words = sentence.split(' ') # space is the delimiter used
count = len(words)
print(count)
```

`split` is a string method that divides a string along a delimiter — one or more characters marking where to split. It returns a list of the split substrings. Printing `words` gives:

OUTPUT

```
['this', 'sentence', 'is', 'false']
```

Another example:

PYTHON

```
comma_words = 'one,two,three,four'
numbers = comma_words.split(',')
print(numbers)
```

OUTPUT

```
['one', 'two', 'three', 'four']
```

Here `','` is the delimiter. Delimiters are not limited to single characters — they can be any string:

PYTHON

```
some_string = 'allISwell'
words = some_string.split('IS')
print(words)
```

OUTPUT

```
['all', 'well']
```

join

Just as we went from a string to a list, we can go from a list of strings back to a string. Consider:

Accept a sequence of words as input and construct a sentence out of it.

A solution without lists:

PYTHON

```
words = ['this', 'sentence', 'is', 'false']
sentence = ''
for word in words:
    sentence += word + ' '
print(sentence)
```

This looks correct but is off by one character. Print the last character:

PYTHON

```
print(sentence[-1])
```

Instead of `e`, it is a space — an extra trailing space. This seems trivial, but programming demands precision. A better version:

PYTHON

```
words = ['this', 'sentence', 'is', 'false']
sentence = words[0]
for word in words[1 : ]:
    sentence += ' ' + word
print(sentence)
```

This is more accurate but clumsy, iterating from the second word. The cleanest solution uses a simple, sophisticated method:

PYTHON

```
words = ['this', 'sentence', 'is', 'false']
sentence = ' '.join(words)
print(sentence)
```

"Isn't that a thing of beauty!" Where `split` chops a string along a delimiter, `join` stitches list strings together using a chosen "thread" — a space here. We could use another string, like a comma:

PYTHON

```
words = ['one', 'two', 'three']
sentence = ','.join(words)
print(sentence)
```

OUTPUT

```
one,two,three
```

That stitching is too tight — let's add some space:

PYTHON

```
words = ['one', 'two', 'three']
sentence = ', '.join(words)
print(sentence)
```

Note the space after the comma.

OUTPUT

one, two, three

Lesson 5.5 — Nested Lists, Matrices, Shallow and Deep Copy

Nested Lists

This lesson revisits the `runs` list created earlier using the `random` library:

PYTHON

```
import random
runs = random.choices([0, 1, 2, 3, 4, 5, 6],
                      weights = [30, 30, 20, 5, 10, 0, 5],
                      k = 120)
assert len(runs) == 120
```

An `assert` statement lets you check whether part of your code behaves as expected. Here, it verifies the list length is `120`, which matters because later computations rely on it. When the expression after `assert` evaluates to `True`, execution moves to the next line; if it is `False`, an `AssertionError` is raised.

Here is an alternate way to organize the same information:

PYTHON

```
overs = list()
new_over = list()
for ball, run in enumerate(runs):
    new_over.append(run)
    if (ball + 1) % 6 == 0:
        overs.append(new_over)
        new_over = list()
```

`overs` is a nested list — a list of lists. Each entry in `overs` stands for an over in the match, holding the runs from that over. The following snippet checks that the outer and inner lists have sizes 20 and 6:

PYTHON

```
assert len(overs) == 20
for over in overs:
    assert len(over) == 6
```

To find the runs on the fourth ball of the third over:

PYTHON

```
answer = overs[2][3]  # zero-indexing
print(answer)
```

The first index refers to the outer list, the second to the inner list. To make this clearer, try:

PYTHON

```

third_over = overs[2]
print(third_over)
fourth_ball = third_over[3]
print(fourth_ball)
assert fourth_ball == overs[2][3]

```

Matrices

Matrices are 2D objects that can be represented as nested lists. First, build a 3×3 matrix filled with zeros:

PYTHON

```

mat = [ ]
for i in range(3):
    mat.append([ ])    # we are appending an empty list
    for _ in range(3):
        mat[i].append(0)
print(mat)

```

OUTPUT

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Notice the inner loop variable `_`. Since it is never actually used, the convention is to name such placeholder variables `_`, existing only to satisfy the language's syntax. Now build another matrix:

PYTHON

```

mat = [ ]
num = 1
for i in range(3):
    mat.append([ ])
    for _ in range(3):
        mat[i].append(num)
        num += 1
print(mat)

```

OUTPUT

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

The same matrix can also be constructed like this:

PYTHON

```

mat = [ ]
num = 1
for _ in range(3):
    row = [ ]
    for _ in range(3):
        row.append(num)
        num += 1
    mat.append(row)
print(mat)

```

Shallow and Deep Copy

Look at this code:

PYTHON

```
mat1 = [[1, 2], [3, 4]]
mat2 = mat1
mat2.append([5, 6])
print(mat1)
print(mat2)
print(mat1 is mat2)
```

Since lists are mutable, `mat2` is merely another name for `mat1`; both refer to the same object, so changing one changes both. Earlier, three techniques for copying lists were shown so edits stay independent. Trying one:

PYTHON

```
mat2 = mat1.copy()
mat2.append([5, 6])
print(mat1)
print(mat2)
print(mat1 is mat2)
```

That works fine. But now try:

PYTHON

```
mat1 = [[1, 2], [3, 4]]
mat2 = mat1.copy()
mat2[0][0] = 100
print(mat1)
print(mat2)
```

OUTPUT

```
[[100, 2], [3, 4]]
[[100, 2], [3, 4]]
```

Here `mat1` changed too! Were we not relying on `copy` to prevent this? The issue is a mutable object nested inside another mutable object. In that scenario `copy` performs only a shallow copy — just a fresh outer list is made, while the inner lists remain shared between `mat1` and `mat2`:

PYTHON

```
print(mat1[0] is mat2[0])
print(mat1[1] is mat2[1])
```

Both print `True`. To copy so that both outer and inner lists become brand-new objects, use `deepcopy`:

PYTHON

```
from copy import deepcopy
mat1 = [[1, 2], [3, 4]]
mat2 = deepcopy(mat1)
mat2[0][0] = 100
print(mat1)
print(mat2)
```

OUTPUT

```
[[1, 2], [3, 4]]
[[100, 2], [3, 4]]
```

Now the two are entirely separate objects:

PYTHON

```
from copy import deepcopy
mat1 = [[1, 2], [3, 4]]
mat2 = deepcopy(mat1)
print(mat1 is not mat2)
print(mat1[0] is not mat2[0])
print(mat1[1] is not mat2[1])
```

All three print `True`. `deepcopy` comes from the `copy` library. Without getting into its inner workings, the takeaway is: when working with nested lists or any collection of mutable objects, reach for `deepcopy` to get a clean, independent copy.

Lesson 5.6 — Tuples: Introduction, Packing and Unpacking

Introduction

A tuple is an immutable sequence of values:

PYTHON

```
family = ('father', 'mother', 'child')
print(type(family))
print(isinstance(family, tuple))
```

Tuples closely resemble lists and support indexing and slicing:

PYTHON

```
print(family[0])
print(family[:2])
```

The key difference is that tuples are immutable and cannot be updated in-place. The following raises an error:

PYTHON

```
##### Alarm! Wrong code snippet! #####
numbers = ('one', 'two', 'four')
numbers[2] = 'three'
##### Alarm! Wrong code snippet! #####
```

This produces a `TypeError` reading `TypeError: 'tuple' object does not support item assignment`. Because of immutability, you cannot append, insert, or delete elements. Only two methods exist for tuples — `count` and `index` — with their usual meanings:

PYTHON

```
numbers = (1, 2, 3, 1, 1)
print(numbers.count(1))
print(numbers.index(2))
```

You can iterate over a tuple with `for`:

PYTHON

```
for num in (1, 2, 3):
    print(num)
```

Since tuples are immutable, they are passed by value like other immutable types (strings, numbers). Useful functions that operate on tuples include `sum`, `max`, and `min`.

More on Tuples

A few additional points.

A singleton tuple needs a trailing comma:

PYTHON

```
i_am_single = (1, )
print(len(i_am_single))
print(isinstance(i_am_single, tuple))
```

Removing the comma changes the result — it becomes an integer:

PYTHON

```
i_am_single = (1)
print(isinstance(i_am_single, int))
```

A list can be converted to a tuple and back:

PYTHON

```
a_list = [1, 2, 3]
a_tuple = tuple(a_list)
b_tuple = (1, 2, 3)
b_list = list(b_tuple)
```

A tuple can hold a non-homogeneous sequence:

PYTHON

```
a_tuple = (1, 'cool', True)
```

Membership is checked with `in`:

PYTHON

```
1 in (1, 2, 3)
'hello' not in ('some', 'random', 'sequence')
```

Tuples can be nested:

PYTHON

```
a = ((1, 2, 3), (4, 5, 6))
print(a[0][2])
```

A tuple can hold mutable objects:

PYTHON

```
a_tuple = ([0, 1, 2], [4, 5, 6])
a_tuple[0][0] = 100
```

This runs without error. Although the tuple itself is immutable, the element inside is mutable. The sequence of objects is not being changed — `a_tuple[0]` still points to the same object. This can be verified:

PYTHON

```
a_tuple = ([0, 1, 2], [4, 5, 6])
print(id(a_tuple[0]))
a_tuple[0][0] = 100
print(id(a_tuple[0]))
```

The `id` of the element remains unchanged. The identities of the objects making up a tuple can never change, but if those objects are mutable (like lists), their internal values may change while retaining their identities.

Lists and Tuples

A summary comparing the two:

LIST	TUPLE
Mutable	Immutable
<code>L = [1, 2, 3]</code>	<code>T = (1, 2, 3)</code>
Supports indexing and slicing	Supports indexing and slicing
Supports item assignment	Doesn't support item assignment
Methods: <code>count</code> , <code>index</code> , <code>append</code> , <code>insert</code> , <code>remove</code> , <code>pop</code> and others	Methods: <code>count</code> , <code>index</code>
To get a list: <code>list(obj)</code>	To get a tuple: <code>tuple(obj)</code>

An example task explores their partnership:

"Populate a list that contains all ordered pairs of positive integers whose product is 100."

Note that order matters — (2, 50) and (50, 2) are treated as different pairs.

PYTHON

```

pairs = [ ]
for a in range(1, 101):
    for b in range(1, 101):
        if a * b == 100:
            pairs.append((a, b))
print(pairs)

```

Here `pairs` is a list of tuples. A tuple is the better choice for each pair because the two elements have a well-defined relationship that should not be accidentally modified.

Packing and Unpacking

Tuples occupy a significant place in Python. Consider:

PYTHON

```

T = 1, 2, 3
print(T)
print(isinstance(T, tuple))

```

Though the first line might look like an error, it works: `T` becomes the tuple `(1, 2, 3)`. This is called **tuple packing**. The reverse is **sequence unpacking**:

PYTHON

```

x, y, z = T
print(x, y, z)

```

Here `T` is unpacked into `x`, `y`, and `z` — the principle behind multiple assignment. The Python documentation states:

"Multiple assignment is a combination of tuple packing and sequence unpacking."

PYTHON

```

x, y, z = 1, 2, 3

```

Above, the RHS is first packed into a tuple, then unpacked into the variables. The "sequence" qualifier exists because any sequence can be unpacked:

PYTHON

```

l1, l2, l3, l4 = 'good'      # string
num1, num2, num3 = [1, 2, 3]  # list
b1, b2 = (True, False)       # tuple
x, y, z = range(3)           # range

```

The same operations apply when functions return multiple values:

PYTHON

```
def max_min(a, b):  
    if a > b:  
        return a, b  
    return b, a  
  
x = max_min(1, 2)  
print(x)  
print(isinstance(x, tuple))
```

Here `x` is a tuple. In the return statements, multiple values are packed into a tuple, so the function essentially returns a tuple.

6

Dictionaries and Sets

Lesson 6.1 — Dictionaries: Introduction, Iterating, Growing, Mutability

Introduction

Suppose we want to store information mapping countries to their capitals in Python:

COUNTRY	CAPITAL
Brazil	Brasilia
Russia	Moscow
India	New Delhi
China	Beijing
South Africa	Cape Town

South Africa has three capitals but only the legislative capital is listed here for convenience, and these five nations form part of the BRICS block.

A dictionary is one of the most interesting data structures in Python — essentially a look-up table. Here is how the BRICS data would be stored:

PYTHON

```
brics = {  
    'Brazil': 'Brasilia',  
    'Russia': 'Moscow',  
    'India': 'New Delhi',  
    'China': 'Beijing',  
    'South Africa': 'Cape Town'  
}
```

A dictionary is a collection of key-value pairs. In this example, each country is a key mapped to its capital (the value) — so `'India'` is the key and `'New Delhi'` is the value. A dictionary object is of type `dict`.

PYTHON

```
print(type(brics))  
print(isinstance(brics, dict))
```

To access the value for a given key:

PYTHON

```
print(brics['India'], 'is the capital of', 'India')  
print(brics['China'], 'is the capital of', 'China')
```

The value for a key can be updated:

PYTHON

```
# Moving to a different capital for South Africa
brics['South Africa'] = 'Pretoria'
# Or we could also store all three capitals
brics['South Africa'] = ('Pretoria', 'Cape Town', 'Bloemfontein')
```

New key-value pairs can also be added. The lesson expands beyond BRICS by creating a new dictionary named `globe` starting as a copy of `brics`, using the `copy()` method (similar to the one for lists):

PYTHON

```
brics = {
    'Brazil': 'Brasilia',
    'Russia': 'Moscow',
    'India': 'New Delhi',
    'China': 'Beijing',
    'South Africa': 'Cape Town'
}
globe = brics.copy()
globe['Spain'] = 'Madrid'
```

Adding a new pair is as simple as the last line above. Keys of a dictionary are unique — the same key cannot map to two different values — but two different keys can share the same value:

PYTHON

```
some_dict = {'key_1': 0, 'key_2': 0}
```

Accessing a key that is not present raises a `KeyError`:

PYTHON

```
##### Alarm! Wrong code snippet! #####
some_dict = {'0': 'zero', '1': 'one'}
print(some_dict[0])
##### Alarm! Wrong code snippet! #####
```

More Examples

A dictionary key can be any immutable object (with a catch discussed later). Examples with basic types `int`, `str`, `float`, `bool`:

PYTHON

```
# int <> int
squares = {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# str <> int
months = {'Jan': 31, 'March': 31, 'May': 31, 'Nov': 30}
# int <> str
roll_numbers = {1: 'CS001', 2: 'CS002', 3: 'CS003'}
# str <> str
names = {'Rohit': 'Sharma', 'Saina': 'Nehwal'}
# str <> float
constants = {'pi': 3.14, 'e': 2.71}
# float <> str
fractions = {0.5: 'half', 0.25: 'quarter', 0.3: 'one-third'}
# int <> bool
binary = {0: True, 1: False}
```

Dictionaries with `list` and `tuple` values:

PYTHON

```
# str <> list
outcomes = {'IND VS AUS': ['IND', 'AUS', 'IND', 'IND'], 'IND VS ENG': ['IND', 'ENG']}
# float <> tuple
bounds = {1.7: (1, 2), 4.3: (4, 5), -1.2: (-2, -1)}
```

Tuples can be keys as long as they contain no mutable objects:

PYTHON

```
# tuple <> list
T1, T2 = (0, 1), (1, 2)
random_numbers = {T1: [0.1, 0.4, 0.9], T2: [1.1, 1.9]}
```

A richer mixed example:

PYTHON

```
# mixed
report_card = {
    'name': 'Ramanujan',
    'age': 18,
    'school': 'KV',
    'marks': (75, 80, 60, 95, 100)
}
```

More on Keys

Saying keys must be immutable is not entirely accurate. Using a list as a key:

PYTHON

```
##### Alarm! Wrong code snippet #####
some_list = [0, 1]
bad_dict = {some_list: 0}
##### Alarm! Wrong code snippet #####
```

This raises a `TypeError` with the message `unhashable type: 'list'`. The error refers to hashability rather than immutability. The more accurate rule:

"The keys of a dictionary must be hashable."

Hash Tables

Python dictionaries are implemented using a hash table. Think of a book-rack with numbered rows, where key-value pairs are books. To find a book you need its row number, which is provided by a *hash function*, denoted h , that converts the key to the row number.

The hash function takes a key k and returns $h(k)$, the *hash value* — analogous to the rack number. Once the rack number is known, the stored item can be retrieved.

An object is hashable if it has a hash value that never changes during its lifetime and can be compared to other objects. Most immutable objects seen so far (`int`, `float`, `str`, `bool`) are hashable, while mutable containers like lists are not. But not every immutable object works as a key:

PYTHON

```
##### Alarm! Wrong code snippet #####
some_tuple = ([0, 1], [2, 3])
bad_dict = {some_tuple: 0}
##### Alarm! Wrong code snippet #####
```

Although `some_tuple` is immutable, it contains lists which are mutable. Per the Python docs, immutable containers are hashable only if their elements are also hashable — so this tuple is unhashable and cannot be a key.

Iterating over Dictionaries

Iterating over keys:

PYTHON

```
squares = {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}    # key is a number, value is its square
for key in squares.keys():
    print(f'The square of {key} is {squares[key]}')
```

`squares.keys()` returns a sequence of keys. Python lets us drop the `keys` method:

PYTHON

```
squares = {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}    # key is a number, value is its square
for key in squares:
    print(f'The square of {key} is {squares[key]}')
```

We can also iterate over key-value pairs:

PYTHON

```
squares = {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}    # key is a number, value is its square
for key, value in squares.items():
    print(f'The square of {key} is {value}')
```

Growing a Dictionary

An empty dictionary can be defined in two ways:

PYTHON

```
D1 = dict()
D1[0] = 1
D2 = { }
D2[0] = 1
```

Problem: Create a dictionary from a list of words that maps words to their lengths.

PYTHON

```
words = ['interstellar', 'dunkirk', 'inception', 'tenet']
lengths = dict()
for word in words:
    lengths[word] = len(word)
print(lengths)
```

A piece of trivia is posed: what do the words in the list have in common?

Mutability

Like lists, dictionaries are mutable. This code demonstrates aliasing:

PYTHON

```
dict_1 = {'one': 1, 'two': 2, 'three': 3}
dict_2 = dict_1
dict_2['four'] = 4
print(dict_1, dict_2)
print(dict_1 is dict_2)
```

Here `dict_2` is an alias of `dict_1`, both pointing to the same object. To make a new dict with the same contents, use `copy()` or the `dict` built-in:

PYTHON

```
dict_1 = {'one': 1, 'two': 2, 'three': 3}
dict_2 = dict_1.copy()      # dict(dict_1) also works
dict_2['four'] = 4
print(dict_1, dict_2)
print(dict_1 is not dict_2)
```

The last line prints `True`, confirming two different objects, so modifying one does not affect the other. But `copy()` only makes a shallow copy — fine for immutable values, but problematic with mutable ones:

PYTHON

```
dict_1 = {'one': [1], 'two': [1, 1], 'three': [1, 1, 1]}
dict_2 = dict_1.copy()
dict_2['one'].append(100)
print(dict_1, dict_2)
print(dict_1 is not dict_2)
print(dict_1['one'] is dict_2['one'])
```

Here the value for `'one'` in both dictionaries is affected because `dict_1['one']` and `dict_2['one']` are still the same object. To fix this, use a deep copy:

PYTHON

```
from copy import deepcopy
dict_1 = {'one': [1], 'two': [1, 1], 'three': [1, 1, 1]}
dict_2 = deepcopy(dict_1)
dict_2['one'].append(100)
print(dict_1, dict_2)
print(dict_1 is not dict_2)
print(dict_1['one'] is not dict_2['one'])
```

Lesson 6.2 — Text Processing

This lesson explores text processing using an excerpt from a talk given by Guido. The excerpt discusses how programming languages are how programmers "express and communicate *ideas*" — with the audience being other programmers rather than computers.

Text processing helps analyze text data. Given a piece of text, some basic questions include:

- How many sentences are there in the text?
- How many words are there in the text?
- How many of them are unique?
- Which word appears the most number of times?

These questions are meaningful for tasks like classifying articles into categories such as lifestyle, science and technology, literature, or films. Rather than reading hundreds of articles manually, you can computationally process each one to find the top common words and infer the topic.

We begin by storing the text in a variable:

PYTHON

```
text = "In reality, programming languages are how programmers express and communicate ideas – and the audience for those ideas"
```

Number of sentences

Sentences may end with a full stop, exclamation mark, or question mark. For simplicity, assume all sentences end with a full stop, so we split on that delimiter:

PYTHON

```
sentences = text.split('.')
```

To inspect the list, temporary printing code is used — understanding what your code does by printing variable contents is important:

PYTHON

```
# Prints one sentence in each line
for sentence in sentences:
    print(sentence)
print(f'There are {len(sentences)} sentences in this text.')
```

OUTPUT

```
In reality, programming languages are how programmers express and communicate
ideas – and the audience for those ideas is other programmers, not computers
The reason: the computer can take care of itself, but programmers are always
working with other programmers, and poorly communicated ideas can cause ex-
pensive flops
In fact, ideas expressed in a programming language also often reach the end
users of the program – people who will never read or even know about the pro-
gram, but who nevertheless are affected by it
There are 4 sentences in this text.
```

Although there are only three sentences, the output reports four. The issue is that `sentences[-1]` is an empty string. When splitting on a delimiter present in the string, substrings are generated on either side; since the full stop is the last character, the substring to its right is empty. The fix removes empty strings:

PYTHON

```
while '' in sentences:
    sentences.remove('')
print(f'There are {len(sentences)} sentences in this text.')
```

OUTPUT

```
There are 3 sentences in this text.
```

One problem solved!

Number of words

To count words, we split each sentence by space:

PYTHON

```
words = [ ]
for sentence in sentences:
    words_ = sentence.split(' ') # words_ contains words in sentence
    words.extend(words_)         # words is the collection of all words
```

Printing `len(words)` gives 86, but wordcounter.net reports 82. To investigate, each word and its index are printed:

PYTHON

```
for index, word in enumerate(words):
    print(index, word)
```

The problematic entries found were:

OUTPUT

```
11 –
23
49
67 –
```

Indices 11 and 67 are em dashes (—), while 23 and 49 are empty strings. Since there are two characters to remove, the list is cleaned:

PYTHON

```
proc_words = [ ]
for word in words:
    if not(word == '' or word == '-'):
        proc_words.append(word)
print(f'There are {len(proc_words)} words in this text')
```

This yields 82 words as expected. One more problem solved!

Number of Unique Words

This section introduces a dictionary to track unique words along with their frequency:

PYTHON

```
uniq_words = dict()
for word in proc_words:
    if word not in uniq_words:
        uniq_words[word] = 0
    uniq_words[word] += 1
print(f'There are {len(uniq_words)} unique words in this text')
```

The result is 62 unique words. But manual inspection shows "programmers" occurs four times. Checking the dict:

PYTHON

```
print(uniq_words['programmers'])
```

The output is 2 — another wrong answer. Making mistakes is the norm, not the exception, and "An error in the code is hidden knowledge." To investigate, we search `proc_words` for entries containing "programmers":

PYTHON

```
for word in proc_words:
    if 'programmers' in word:
        print(word)
```

OUTPUT

```
programmers
programmers,
programmers
programmers,
```

The problem is the comma special character. Another issue is capitalization at the beginning of sentences. The code is revised to handle both:

PYTHON

```

proc_words = [ ]
for word_ in words:
    word = word_.lower()
    if not(word == '' or word == '-'):
        if not word_.isalnum():
            word = word_[:-1]
        proc_words.append(word)
print(f'There are {len(proc_words)} words in this text')

```

Here, each word is converted to lowercase, em dashes and empty strings are ignored, and if a word contains a special character it is stripped of the trailing character (assuming special characters appear at the end). Examples in the text: "programmers," and "reason:". Then

`uniq_words` is regenerated:

PYTHON

```

uniq_words = dict()
for word in proc_words:
    if word not in uniq_words:
        uniq_words[word] = 0
    uniq_words[word] += 1
print(f'There are {len(uniq_words)} unique words in this text')

```

Now there are 58 unique words. This can be verified by printing all words and counts:

PYTHON

```

for word, freq in uniq_words.items():
    print(word, freq)

```

As a test, the sum of counts should equal the total number of words:

PYTHON

```

total = 0
for word in uniq_words:
    total += uniq_words[word]
assert total == len(proc_words)

```

Since no `AssertionError` is raised, the result is correct.

Frequent Words

The final task is finding the top three most frequently occurring words:

PYTHON

```
first_word = second_word = third_word = ''
first_val = second_val = third_val = 0
for word, freq in uniq_words.items():
    if freq > first_val:
        first_val, second_val, third_val = freq, first_val, second_val
        first_word, second_word, third_word = word, first_word, second_word
    elif freq > second_val and freq < first_val:
        second_val, third_val = freq, second_val
        second_word, third_word = word, second_word
    elif freq > third_val and freq < second_val:
        third_val = freq
        third_word = word
print(first_word, first_val)
print(second_word, second_val)
print(third_word, third_val)
```

OUTPUT

```
the 6
programmers 4
in 3
```

"Programmers" is the second most frequent word, while "the" and "in" are first and third. These common words are called stop-words; removing them makes "programmers" the most frequent non-trivial word. So without reading the text, one could guess it is about programmers — thanks to Python.

Summary

The main lesson takeaway is the kinds of mistakes made and how each was fixed. In nearly every problem, we started with a solution, tested it, discovered something was wrong, and went back to fix the problem.

Lesson 6.3 — Pangrams and Dictionary Methods

Pangrams and Dictionaries

This lesson explores building a mapping between the 26 letters of the English alphabet and the numbers 1 through 26, where each letter maps to a unique number (a→1, b→2, ... z→26) in a simple linear fashion.

The most tedious approach is to type out the entire dictionary by hand:

PYTHON

```

mapping = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5,
          'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10,
          'k': 11, 'l': 12, 'm': 13, 'n': 14, 'o': 15,
          'p': 16, 'q': 17, 'r': 18, 's': 19, 't': 20,
          'u': 21, 'v': 22, 'w': 23, 'x': 24, 'y': 25,
          'z': 26}

for letter, count in mapping.items():
    print(letter, count)

```

Typing this out is dull and mechanical, and copy-pasting is impractical if you do not want to open the textbook every time you need this mapping. A more interesting method uses a **pangram** — a sentence that uses every letter of the alphabet:

the quick brown fox jumps over the lazy dog

PYTHON

```

pangram = 'the quick brown fox jumps over the lazy dog'
words = pangram.split(' ')           # get list of words in the sentence
letters = ''.join(words)             # join the words back; eliminates spaces
sorted_letters = sorted(letters)     # sort letters
mapping, count = dict(), 0
for letter in sorted_letters:
    # check if letter is not present in dict
    # to avoid counting same letter multiple times
    if letter not in mapping:
        count += 1
        mapping[letter] = count      # map the letter to count
for letter, count in mapping.items():
    print(letter, count)

```

There is much to learn from these 14 lines. With this dictionary in place, the lesson moves into dictionary methods bundled with `dict`.

Dictionary Methods

`keys` and `items` were seen earlier. Both are methods returning a view object that can be iterated over. Quoting the Python documentation, view objects provide a dynamic view on the dictionary's entries, so when the dictionary changes, the view reflects those changes.

PYTHON

```

keys = mapping.keys()
print(keys)

```

OUTPUT

```
dict_keys(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'])
```

Using `list()`, both the `keys` and `items` views can be turned into lists:

PYTHON

```
keys_list = list(mapping.keys())
print(keys)
items_list = list(mapping.items())
print(items)
```

OUTPUT

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o',
 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
[('a', 1), ('b', 2), ('c', 3), ('d', 4), ('e', 5), ('f', 6), ('g', 7), ('h',
 8), ('i', 9), ('j', 10), ('k', 11), ('l', 12), ('m', 13), ('n', 14), ('o',
 15), ('p', 16), ('q', 17), ('r', 18), ('s', 19), ('t', 20), ('u', 21), ('v',
 22), ('w', 23), ('x', 24), ('y', 25), ('z', 26)]
```

Here `keys_list` is a list of the dictionary's keys, while `items_list` is a list of tuples, each being a key-value pair. Another useful method is `values`, which returns a **view** on the values:

PYTHON

```
view = mapping.values()
view_list = list(view)
```

All three views — `keys`, `items`, and `values` — support membership tests:

PYTHON

```
print('a' in mapping.keys())
print(1 in mapping.values())
print(('a', 1) in mapping.items())
```

All three of these return `True`. Membership tests for keys can be written more simply:

PYTHON

```
print('a' in mapping)
print('x' in mapping)
print('ab' not in mapping)
```

Notice that dropping the `keys` method still works. To delete a key from a dictionary, use the familiar `pop` method:

PYTHON

```
mapping['ab'] = 3          # some noise added to mapping
value = mapping.pop('ab')
print(value)
print('ab' not in mapping)
```

If `key` is a key in dictionary `D`, then `D.pop(key)` removes that key and returns its associated value. Removing a key also removes its value. Dictionaries are "aristocratic data structures": keys sit higher in the hierarchy, and values depend on keys for their existence.

Lesson 6.4 — Dictionaries in Action: LMS

This lesson explores how a Learning Management System (LMS) works — the software that powers an online degree portal. It aims to answer how assignment submissions get recor-

ded and graded. At a high level, an LMS consists of two components: a frontend and a backend.

As a user, you interact with the frontend — the website displaying content. When you take an action (like clicking submit on a graded assignment), that action goes to the backend as input. The backend processes it and returns output to the frontend, which displays the result. Python plays a prominent role in the backend.

Grading requires two inputs — the assignment and the corresponding submission — and returns a result as output. The grader can be expressed as a function:

PYTHON

```
def grader(assignment, submission):  
    """Grading logic"""  
    result = 0.0  
    return result
```

This function is incomplete; we still need to model an assignment and its submission.

Assignment Model

An assignment is essentially a list of problems, so modeling an assignment reduces to modeling a problem. A problem could have these attributes:

ATTRIBUTE	TYPE
id	string
question	string
type	string
options	list
answers	tuple
marks	float

For grading, only two attributes are needed: the problem-id and the answers. The assignment becomes a list of dictionaries:

PYTHON

```
# assume that the assignment has three problems  
# the assignment will be a list of dictionaries  
assignment = [  
    {'id': '10001', 'answers': (0, 1), 'marks': 2.0},  
    {'id': '10002', 'answers': (1, ), 'marks': 1.0 },  
    {'id': '10003', 'answers': (2, ), 'marks': 2.0}  
]
```

NOTE

A singleton tuple is written as `(<item>, )` — the comma cannot be omitted.

Several attributes from the table do not appear in the dictionary because they are not relevant to grading; they were listed to give a fuller picture of how assignments can be modeled.

Submission Model

The submission model is more involved. It has global attributes like the user's name, roll number, and submission time, plus local attributes such as the options selected per problem.

ATTRIBUTE	TYPE
name	string
roll_number	string
timestamp	string
problems	list

A sample submission:

PYTHON

```
submission = {
    'name': 'Kapil Dev',
    'roll_number': 'BSC1001',
    'time': 'Sunday 18 April 2021 10:23:30 PM IST',
    'problems': [
        {'id': '10001', 'selected': (0, 1)},
        {'id': '10002', 'selected': (1, )},
        {'id': '10003', 'selected': (3, )}
    ]
}
```

The `submission` is a fairly complicated object. It is a dictionary; the first three keys are simple, but the value of `'problems'` is a list of dictionaries. Complexity could increase further since a user might make multiple submissions (a list of submissions), but that is left aside for now.

Grader

Although representing the assignment as a list of dictionaries is not bad, the grader would have to search the list for each problem id when grading. Since problem ids are unique, a better representation is possible:

PYTHON

```

assignment_ = [
    {'id': '10001', 'answers': (0, 1), 'marks': 2.0},
    {'id': '10002', 'answers': (1, ), 'marks': 1.0 },
    {'id': '10003', 'answers': (2, ), 'marks': 2.0}
]
assignment = dict()
for problem in assignment_:
    problem_id = problem['id']
    answers = problem['answers']
    marks = problem['marks']
    assignment[problem_id] = {'answers': answers, 'marks': marks}

```

The assignment now looks like this:

PYTHON

```

assignment = {
    '10001': {
        'answers': (0, 1),
        'marks': 2.0
    },
    '10002': {
        'answers': (1, ),
        'marks': 1.0
    },
    '10003': {
        'answers': (2, ),
        'marks': 2.0
    },
}

```

We can now complete the grader using this new assignment model:

PYTHON

```

def grader(assignment, submission):
    """Grading logic"""
    result = 0.0
    for problem in submission['problems']:
        problem_id = problem['id']
        selected = problem['selected']
        answers = assignment[problem_id]['answers']
        if answers == selected:
            result += assignment[problem_id]['marks']
    return result

```

Lesson 6.5 — Sets

Introduction

A set is an unordered collection with no duplicate elements. Unlike lists and tuples, a set has no notion of order — that is why it is called an unordered collection rather than a sequence. You can define one like this:

PYTHON

```
even_nums = {2, 4, 6, 8, 10}
print(type(even_nums))
print(isinstance(even_nums, set))
```

OUTPUT

```
<class 'set'>
True
```

Note the syntactic similarity between sets and dictionaries — both use curly braces. A dictionary holds key-value pairs while a set holds just values. Python sets closely mirror mathematical sets, so most familiar mathematical set properties carry over naturally.

PYTHON

```
nums_1 = {2, 4, 6, 8, 10}
nums_2 = {2, 2, 4, 4, 6, 6, 8, 8, 10, 10}
print(nums_1, nums_2)
print(nums_1 == nums_2)
print(nums_1 is not nums_2)
```

OUTPUT

```
{2, 4, 6, 8, 10} {2, 4, 6, 8, 10}
True
True
```

Since sets do not support duplicates, `nums_1` and `nums_2` are equal, though they are not the same object. Sets support membership testing like other collections.

PYTHON

```
nums = {1, 2, 3, 4, 5}
print(1 in nums)
print(6 not in nums)
```

The number of elements (its cardinality) is given by `len()` :

PYTHON

```
nums = {1, 2, 3, 4, 5}
print(f'Cardinality of nums is {len(nums)}')
```

Sets cannot be indexed, which makes sense since they are not ordered. This code raises an error:

PYTHON

```
##### Alarm! Wrong code snippet! #####
some_set = {'this', 'is', 'a', 'set'}
print(some_set[0])
##### Alarm! Wrong code snippet! #####
```

Any hashable object can be added to a set, including most immutable types like `int`, `float`, `str`, and `tuple`. Caveat: a tuple containing lists is unhashable and cannot be added.

PYTHON

```
a_set = {1.0, 'one', 1, True, (1, )}    # valid set
not_a_set = {[1, 2], [3, 4]}           # not a valid set
```

`not_a_set` raises a `TypeError` as expected.

Iterating through Sets

Although a set is not a sequence, you can still iterate over its elements.

PYTHON

```
nums = {1, 2, 3, 4, 5}
for num in nums:
    print(num)
```

Growing Sets

How do we define an empty set?

PYTHON

```
##### Alarm! Be careful about the variable name! #####
empty_set = { }
print(isinstance(empty_set, set))
print(isinstance(empty_set, dict))
##### Alarm! Be careful about the variable name! #####
```

As it turns out, `empty_set` is actually an empty dictionary — `{ }` still denotes an empty dictionary as it did in earlier lessons. So the correct way to make an empty set is:

PYTHON

```
empty_set = set()
print(isinstance(empty_set, set))
```

With empty sets and iteration available, we can build sets from scratch.

Consider the first 100 powers of 7: $7^1, 7^2, \dots, 7^{100}$. Note the last digit of each.
How many are unique? What are they?

There is a simple mathematical answer, but let's pursue a computational one:

PYTHON

```
num = 1
digits = set()
for i in range(100):
    num *= 7
    last = num % 10
    digits.add(last)
print(digits)
```

`add` inserts elements into a set. Sets are ideal when duplicates crop up often and you do not care about them. The same task can be done with lists:

PYTHON

```

num = 1
digits = [ ]
for i in range(100):
    num *= 7
    last = num % 10
    if last not in digits:
        digits.append(last)
print(digits)

```

Set Operations

Mathematical sets interact through operations, and Python sets aim to match them:

- Subset
- Superset
- Union
- Intersection
- Difference

Subset: A is a subset of B ($A \subseteq B$) if every element of A is in B .

PYTHON

```

A = {1, 3, 5}
B = {1, 2, 3, 4, 5}
print(A.issubset(B))    # method-1
print(A <= B)           # method-2

```

Both lines return `True`. A *proper* subset ($A \subset B$) requires every element of A to be in B with $A \neq B$:

PYTHON

```

A = {1, 2, 3}
B = {1, 2, 3}
print(A <= B)           # method-1
print(A < B)            # method-2

```

The `A < B` operator checks proper-subset status; here A is not a proper subset, so the second print gives `False`.

Superset: A is a superset of B ($A \supset B$) if every element of B is in A .

PYTHON

```

A = {1, 3, 5}
B = {1, 2, 3, 4, 5}
B.issuperset(A)         # method-1
print(B >= A)           # method-2

```

Union: the union ($A \cup B$) contains elements in either A or B or both.

PYTHON

```

A = {1, 3, 5}
B = {2, 4, 6}
C1 = A.union(B)      # method-1
C2 = A | B           # method-2
print(C1, C2)
print(C1 == C2)

```

For multiple sets:

PYTHON

```

A1, A2, A3, A4 = {1}, {2, 3}, {4, 5, 6}, {7, 8, 9, 10}
B1 = A1.union(A2, A3, A4)  # method-1
B2 = A1 | A2 | A3 | A4     # method-2
print(B1, B2)
print(B1 == B2)

```

Intersection: the intersection ($A \cap B$) contains elements common to both.

PYTHON

```

A = {2, 4, 6}
B = {2, 4}
C1 = A.intersection(B)  # method-1
C2 = A & B               # method-2
print(C1, C2)
print(C1 == C2)

```

With no common elements, we get the empty set:

PYTHON

```

even, odd = {2, 4, 6}, {1, 3, 5}
common = even & odd
assert common == set()

```

The `assert` here adds variety; since no `AssertionError` is raised, the result is correct.

Difference: the difference of A and B ($A - B$ or $B - A$) are not the same.

- $A - B$: elements in A not in B .
- $B - A$: elements in B not in A .

PYTHON

```

A = {1, 2, 3, 4}
B = {2, 4, 5}
C1 = A.difference(B)  # method-1
C2 = A - B            # method-2
print(C1, C2)
print(C1 == C2)
D1 = B.difference(A)  # method-1
D2 = B - A            # method-2
print(D1, D2)
print(D1 == D2)

```

Other Set Methods

Now for methods with a more computational character. To remove an element, use

`remove` :

PYTHON

```
A = {'this', 'is', 'a', 'set'}
print('Before', A)
A.remove('this')
print('After', A)
```

Removing an absent element raises a `KeyError` :

PYTHON

```
A = {'this', 'is', 'a', 'set'}
A.remove('cool')    # error!
```

Consider this problem:

Given a list `L`, extract all unique elements into another list `L_uniq`. Order does not matter.

A solution without sets:

PYTHON

```
L = [1, 2, 3, 3, 4, 5, 6, 1, 2, 2]
L_uniq = [ ]
for elem in L:
    if elem not in L_uniq:
        L_uniq.append(elem)
print(L_uniq)
```

And with sets:

PYTHON

```
L = [1, 2, 3, 3, 4, 5, 6, 1, 2, 2]
S = set(L)
L_uniq = list(S)
print(L_uniq)
```

Passing a list to `set` strips duplicates and yields the unique elements.

Mutability

Sets are mutable.

PYTHON

```
A = {1, 2, 3}
B = A
B.add(4)
print(A, B)
print(A is B)
```

Here `A` and `B` refer to the same object. As before, there are two ways to make a shallow copy:

PYTHON

```
A = {1, 2, 3}
B1 = A.copy()
B2 = set(A)
B1.add(4)
B2.add(0)
print(A, B1, B2)
print(A is not B1)
print(A is not B2)
```

7

File Handling

Lesson 7.1 — Why Files

Why files

This lesson motivates the concept of files by comparing computer memory to human memory. Just as humans can only hold a limited number of "chunks" of information in short-term memory (often cited as around seven), computers also have finite short-term memory despite being powerful. The solution for both is external storage. Files serve computers the way books serve humans, providing a permanent place to record information that can be retrieved whenever needed.

File handling

We typically open files by double-clicking an icon. Consider an example file containing income-expenditure data for a family over five months:

```
OUTPUT
Income      Expenditure
12,000      10,000
50,000      45,000
75,000      35,000
14,000      12,000
60,000      40,000
```

The goal is to create a new file that adds a third column for savings, producing the following output:

```
OUTPUT
Income      Expenditure  Savings
12,000      10,000      2,000
50,000      45,000      5,000
75,000      35,000      40,000
14,000      12,000      2,000
60,000      40,000      20,000
```

The task appears simple — open the file, calculate the values, and paste them in a new column. But there is a problem of scale: what if the number of entries grows dramatically? For instance, processing data for every family in a neighborhood. With 10 years of data for 1000 families, we would be dealing with $1000 * 10 * 12 = 120,000$ entries, which would overwhelm both a calculator and a person.

This is where Python helps. We can write just a few lines of code to automate the entire process. Upcoming lessons will cover file processing, teaching these operations:

- opening a file and closing it
- reading from a file
- writing to a file

"File handling is an umbrella term that denotes all these operations."

Lesson 7.2 — Creating, Opening, Reading and Writing a File

Creating a file in Replit

This lesson uses Replit and the `Add File` button to create a file. Each file needs a name — here it is called `examples.txt`. The following lines are added to the file:

```
OUTPUT
one
two
three
four
five
```

After creating the file, it appears in Replit when you click on `examples.txt`.

A `.txt` file is a text file, identifiable by the extension at the end of the file name. Different files use different extensions. For example, `main.py` is a file with the `py` extension, which is why it appears alongside `examples.txt` under the Files tab in Replit.

Opening and reading from a file

To open the file and print its contents to the console, head to `main.py` and type:

```
PYTHON
f = open('examples.txt', 'r')
for line in f:
    print(line)
f.close()
```

`open()` is a built-in function accepting two arguments:

- file name
- mode

The first argument is the file name (`'examples.txt'`). The second is the mode — here `'r'` for read mode. Both arguments are strings. The `open()` function returns a file object, called `f` in the code. The loop goes through each line and prints it. Finally, `close()` closes the file. "It is a good practice to close the file once we are done with processing it."

OUTPUT

```
one

two

three

four

five
```

There is an extra blank line between successive lines. To suppress these, modify the print function:

PYTHON

```
f = open('examples.txt', 'r')
for line in f:
    print(line, end = '')    # there is NO SPACE between the quotes
f.close()
```

By default, `print()` appends a newline character (`'\n'`). Using `end = ''` appends the empty string instead, removing the extra line.

OUTPUT

```
one
two
three
four
five
```

Opening and writing to a file

Consider this code:

PYTHON

```
f = open('writing.txt', 'w')
f.write('one ')
f.write('two ')
f.write('three ')
f.write('four ')
f.write('five')
f.close()
```

Here the file is opened in write mode. When executed, it creates a file in Replit called `writing.txt` . The `write()` method writes to the file, taking the content as a string argument. Even though five words are written across five lines of code, all of them get written to the same line in the file. To move to a new line, use the `'\n'` character. Try this code:

PYTHON

```
f = open('writing.txt', 'w')
f.write('one')
f.write('\n')
f.write('two')
f.write('\n')
f.write('three')
f.write('\n')
f.write('four')
f.write('\n')
f.write('five')
f.close()
```

A better way, in fewer lines, is to append `\n` to every line you wish to write:

PYTHON

```
f = open('writing.txt', 'w')
f.write('one\n')
f.write('two\n')
f.write('three\n')
f.write('four\n')
f.write('five')
f.close()
```

This produces the same file with fewer lines of code. The next lesson takes a closer look at the idea of a file object.

NOTE

Notice that no `'\n'` was added after `five`. Consider why that is — and try running the code with `'\n'` after `five`.

Lesson 7.3 — File Object and Mode

File Object

The `open()` function returns a file object. To understand a file object, consider an analogy: imagine you are the CEO of a tech company who is great at multitasking, but overwhelmed by responsibilities. You hire a personal assistant (PA) to manage the workload. If you have a meeting scheduled with delegates from another company at 5:00 PM the following Tuesday, you would instruct your PA to "make a note of this meeting", and they would record it in a file.

Later, when you suddenly remember the meeting, you would ask your PA to fetch the details. In both scenarios, it is the PA who directly interacts with the file — first recording information, then retrieving it.

The file object acts as your PA, mediating between you (the coder) and the file stored on your computer's hard disk. You give instructions to the file object, which handles the actual reading and writing. All communication between you and the file passes through the file object.

Mode

There are two modes for opening a file, covered more thoroughly in later lessons.

Read mode

The mode you choose for processing the file is an instruction that always comes from you and is directed at the file object. When reading, information flows from the file, through the file object, and to you. To read a file, you open it in read mode:

PYTHON

```
f = open('<file_name>', 'r')
# ...
# code for reading something from file
# ...
f.close()
```

Write mode

When writing to a file, information flows from you, through the file object, and to the file. To write to a file, you open it in write mode:

PYTHON

```
f = open('<file_name>', 'w')
# ...
# code for writing something into the file
# ...
f.close()
```

Lesson 7.4 — File Methods

`read()`

Working with `examples.txt` (containing the lines one, two, three, four, five), the `read()` method returns the entire file as a single string.

PYTHON

```
f = open('examples.txt', 'r')
content = f.read()
print(content)
f.close()
```

OUTPUT

```
one
two
three
four
five
```

The variable `content` is one string with newline characters between lines: `'one\ntwo\nthree\nfour\nfive'`. Every line except the last ends in `\n`, which is why printing produces separate lines, and why looping with `print(line)` earlier produced blank lines between entries.

readline()

Reads one line at a time.

PYTHON

```
f = open('examples.txt', 'r')
line1 = f.readline()
line2 = f.readline()
line3 = f.readline()
line4 = f.readline()
line5 = f.readline()
f.close()
```

VARIABLE	VALUE
line1	'one\n'
line2	'two\n'
line3	'three\n'
line4	'four\n'
line5	'five'

Reading past the end returns an empty string, which gives a way to detect the file's end. A loop-based approach:

PYTHON

```
f = open('examples.txt', 'r')
line = f.readline()
while line != '':
    print(line, end = '')
    line = f.readline()
f.close()
```

A more compact version uses truthiness and `strip()`:

PYTHON

```
f = open('examples.txt', 'r')
line = f.readline()
while line:
    print(line.strip())
    line = f.readline()
f.close()
```

Python treats empty sequences as `False`, as this demonstrates:

PYTHON

```
line = ''
if not line:
    print('It works!')
```

readlines()

Reads the file into a list of line strings.

PYTHON

```
f = open('examples.txt', 'r')
lines = f.readlines()
for line in lines:
    print(line.strip())
f.close()
```

The resulting list: `['one\n', 'two\n', 'three\n', 'four\n', 'five']`.

write()

Writing in a loop, but naively adding `\n` to every line creates an extra empty line (six lines instead of five):

PYTHON

```
f = open('writing.txt', 'w')
lines = ['one', 'two', 'three', 'four', 'five']
for line in lines:
    f.write(line + '\n')
f.close()
```

Fixing it by not appending `\n` after the last item:

PYTHON

```
f = open('writing.txt', 'w')
lines = ['one', 'two', 'three', 'four', 'five']
for i in range(len(lines)):
    line = lines[i]
    if i != len(lines) - 1:
        f.write(line + '\n')
    else:
        f.write(line)
f.close()
```

write() only accepts strings. Passing an integer:

PYTHON

```
f = open('writing.txt', 'w')
f.write(1)
f.close()
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    f.write(1)
TypeError: write() argument must be str, not int
```

Convert integers to strings first:

PYTHON

```
f = open('writing.txt', 'w')
f.write(str(1))
f.close()
```

An exercise asks you to run the following (note `writeline` is not a real method) and consider why it fails:

PYTHON

```
f = open('writing.txt', 'w')
f.writeline(str(1))
f.close()
```

writelines()

Writes a list of strings to a file.

PYTHON

```
f = open('writing.txt', 'w')
lines = ['1\n', '2\n', '3\n', '4\n', '5']
f.writelines(lines)
f.close()
```

Resulting file contents:

OUTPUT

```
1
2
3
4
5
```

Lesson 7.5 — Reading CSV Files

CSV files

This section builds on earlier work with simple files by introducing CSV files, which are common in data science. In these files, values on each line are separated by commas. A sample CSV file might contain rows like `col0,col1,col2,col3` followed by data rows such as `row1,item11,item12,item13`.

These files work well for tabular data. The first line is the **header**, describing the columns; the remaining lines are the data rows. Represented as a table:

	COL0	COL1	COL2	COL3
row1	item11	item12	item13	
row2	item21	item22	item23	
row3	item31	item32	item33	
row4	item41	item42	item43	
row5	item51	item52	item53	

Reading a CSV file

The lesson creates a file named `table.csv` in Replit. Opening and reading it works just like a text file:

PYTHON

```
f = open('table.csv', 'r')
for line in f:
    print(line.strip())
f.close()
```

OUTPUT

```
Name,Physics,Mathematics,Chemistry
Newton,100,98,90
Einstein,100,85,88
Ramanujan,70,100,70
Gauss,100,100,70
```

Task: Print each student's chemistry marks, one per line — meaning we extract the last column.

Consider one line. The newline appears on every line except the last, so we strip it:

PYTHON

```
# The '\n' at the end will be present for all lines except the last one
line = 'Newton,100,98,90\n'
line = line.strip() # removes the \n character
```

Split the line on commas using `split()` :

PYTHON

```
line = 'Newton,100,98,90\n'
line = line.strip()
columns = line.split(',')
print(columns)
```

This returns a list of strings:

OUTPUT

```
['Newton', '100', '98', '90']
```

Take the last element and convert it to an integer:

PYTHON

```
line = 'Newton,100,98,90'
line = line.strip()
columns = line.split(',')
chem_marks = int(columns[-1])
print(chem_marks)
```

Now apply this to all rows using a loop:

PYTHON

```
f = open('table.csv', 'r')
for line in f:
    line = line.strip()
    columns = line.split(',')
    chem_marks = int(columns[-1])
    print(chem_marks)
f.close()
```

But this produces an error:

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    chem_marks = int(columns[-1])
ValueError: invalid literal for int() with base 10: 'Chemistry'
```

The problem is that the code tried to convert the header's last column to an integer. When reading CSV files, "we need to find a way to deal with the header." Modified code:

PYTHON

```
f = open('table.csv', 'r')
header = f.readline()
# The file object has finished reading the first line
# It is now ready to read from the second line onwards
for line in f:
    line = line.strip()
    columns = line.split(',')
    chem_marks = int(columns[-1])
    print(chem_marks)
f.close()
```

This works. After reading the header on the second line, the for loop begins from the file's second line onward.

An alternative using `readlines()` alone:

PYTHON

```
f = open('table.csv', 'r')
lines = f.readlines()
# lines[1: ] is the rest of the list
# after ignoring the header
for line in lines[1: ]:
    line = line.strip()           # strip the line of \n
    columns = line.split(',')     # split based on comma
    chem_marks = int(columns[-1]) # convert last column to int
    print(chem_marks)
f.close()
```

`readlines()` is reasonable for small files (under ~1000 lines): all lines come back in a list, so file processing becomes list processing. If `lines` is the list, then `lines[i]` corresponds to the (i + 1)th line in the file, and the ith file line corresponds to `lines[i - 1]`.

PROCESSING LARGE FILES

For large files, `readline()` is the best method, reading one line at a time. Using `readlines()` on large files is dangerous because it dumps the whole file into a list of strings, consuming large amounts of memory.

The same program written with `readline` :

PYTHON

```
f = open('table.csv', 'r')
header = f.readline().strip()      # this is for the header
line = f.readline()                # second line; actual rows begin
while line:
    line = line.strip()             # strip the line of \n
    columns = line.split(',')       # split based on comma
    chem_marks = int(columns[-1])   # convert last column to int
    print(chem_marks)
    line = f.readline()            # read the next line in the file
f.close()
```

Files to Collections

It is often useful to convert a CSV file into a suitable collection. Here the goal is to build a list of dictionaries:

PYTHON

```
data = [
    {'Name': 'Newton', 'Physics': 100, 'Mathematics': 98, 'Chemistry': 90},
    {'Name': 'Einstein', 'Physics': 100, 'Mathematics': 85, 'Chemistry': 88},
    {'Name': 'Ramanujan', 'Physics': 70, 'Mathematics': 100, 'Chemistry': 70},
    {'Name': 'Gauss', 'Physics': 100, 'Mathematics': 100, 'Chemistry': 70}]
```

Points to note:

- It is a list of dictionaries, one dictionary per file row.
- Header elements become the keys in each dictionary.
- Values have different data types — names are strings, marks are integers.

The full code is broken into parts. First, get the stripped list of lines:

PYTHON

```
f = open('table.csv', 'r')
lines = [ ]
for line in f.readlines():
    line = line.strip()
    lines.append(line)
# Now, we have the list of lines
```

Next, extract header details:

PYTHON

```
header = lines[0].split(',')    # Lines[0] is the first row
num_cols = len(header)
# Now, we have the header information
# We also know the number of columns in the file
```

Finally, process the remaining lines:

PYTHON

```
### Block-3 ###
data = [ ]                # List to store the contents
for line in lines[1: ]:   # ignore the header
    cols = line.split(',') # split the column based on comma
    row_dict = dict()      # create a dict to store this particular row
    for i in range(num_cols): # go through each element (column) in this row
        key = header[i]     # key will be this column's name
        if key == 'Name':   # are we storing a name or a number?
            value = cols[i]  # since this is a name, don't convert to int
        else:
            value = int(cols[i]) # since this is marks, convert to int
        row_dict[key] = value # update dict
    data.append(row_dict)    # append this row to the list data
f.close()
```

NOTE

CSV stands for comma-separated values.

8

Object Oriented Programming

Lesson 8.1 — Objects and Classes

Objects and Classes

This lesson introduces the fundamentals of object-oriented programming (OOP), aiming to explain the concept with minimal jargon. It uses the phrase "Unity in diversity" as a way to grasp the idea of objects.

The core intuition: humanity is united (we are all humans sharing this planet), yet each individual is unique — differing in properties like height and weight. This can be framed as two forces: a global force uniting everyone and a local force giving each person a distinct identity. In OOP terms, each human is an **object**, while all humans belong to the **class** "Humanity."

The same logic extends to cars. No two cars are identical — brand and speed are points of difference — yet a car is clearly distinct from a train because of shared global features. Each car is an object, and all cars belong to the class "Car."

Moving from concrete to abstract, here are the key definitions:

"Objects are entities that have certain attributes along with operations associated with them."

For cars, attributes could include speed and fuel level; operations could include start, stop, accelerate, decelerate, and fill fuel tank.

"A class is a blueprint or a template that is used to create objects."

The specification of a car lives in a class like `Car` (note the capital "C"), while a human's specification lives in a class like `Human`. A class is like a Google form: a template created once and distributed, with each person filling it in differently to create different objects.

"Object Oriented Programming (OOP) is a paradigm that looks at the world as a collection of objects and the interactions among them."

OOP in Python: an example

Consider a simple student template with two pieces of information: Name and Marks. The desired operations are to update the student's marks and to print the student's details.

PYTHON

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def update_marks(self, marks):
        self.marks = marks

    def print_details(self):
        print(f'{self.name}')
```

`class` is the Python keyword for defining classes, similar to how `def` defines functions. `Student` is the class name being created. The class contains three functions — `__init__`, `update_marks`, and `print_details`. Functions defined inside a class are called **methods**. Among them, `__init__` has a special role and is called the **constructor**.

To create an object of type `Student` :

PYTHON

```
anish = Student('Anish', 95)
```

Now `anish` is an object of type `Student`. To verify this, run:

PYTHON

```
print(type(anish)) # output should be: <class '__main__.Student'>
```

This example will be revisited in the next lesson to explore various features of a class.

Lesson 8.2 — Classes, Objects, self, Class vs Object Attributes

Classes and Objects

The lesson continues with the `Student` class — do not worry about `self` yet. The class definition:

PYTHON

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def update_marks(self, marks):
        self.marks = marks
    def print_details(self):
        print(f'{self.name}:{self.marks}')
```

An object is created with `anish = Student('Anish', 80)`. Using the class name on the right-hand side invokes the **constructor** (`__init__()`). Its arguments become **attributes** accessed via the `.` operator:

PYTHON

```
print(anish.name)
print(anish.marks)
```

`__init__()`, `update_marks()`, and `print_details()` are **methods** — functions defined in a class. To update marks: `anish.update_marks(95)`. In short, attributes define an object's state, while methods "capture the behaviour of the object."

self

`self` "is used to point to the current object." Given two objects, calling `lakshmi.print_details()` is really executed as `Student.print_details(lakshmi)` — the object is passed as an argument. Likewise `anish.update_marks(95)` equals `Student.update_marks(anish, 95)`. This is why the first parameter of every method is `self`. So `self.name = name` assigns the argument to the current object's attribute.

Class Attributes vs Object Attributes

Object attributes differ per object. To track a student count, use a **class attribute**, shared by all objects:

PYTHON

```
class Student:
    counter = 0
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
        Student.counter += 1
    def update_marks(self, marks):
        self.marks = marks
    def print_details(self):
        print(f'{self.name}:{self.marks}')
```

Creating three students and printing `Student.counter` each time:

OUTPUT

```
Number of students in the program = 1
Number of students in the program = 2
Number of students in the program = 3
```

`print(madhavan.counter)` gives `3` — a class attribute can be accessed by any object. But assigning `madhavan.counter = -1`:

OUTPUT

```
Student counter: 3
Madhavan counter: -1
```

This creates a new object attribute with the same name; Python "prioritizes the object attribute," delinking `madhavan.counter` from the class one. To reset: `Student.counter = 0` then print gives `Student counter: 0`.

The final exercise (adding `Rohan`) produces:

```
OUTPUT
Usha counter: 0
Madhavan counter: -1
Student counter: 1
Usha counter: 1
Madhavan counter: -1
```

Key point: "class attributes cannot be updated by an object" — only referenced or accessed.

Attributes can also be created dynamically at runtime, not only in the constructor:

PYTHON

```
class Student:
    def __init__(self, name):
        self.name = name

anish = Student('Anish')
anish.maths = 100
anish.physics = 90
anish.chem = 70
```

Every `Student` gets `name` at creation, but `maths`, `physics`, and `chem` are unique to `anish`.

Lesson 8.3 — Inheritance and Method Overriding

Inheritance

This lesson returns to the guiding philosophy of object-oriented programming: **"Unity in diversity."** The class embodies unity while objects embody diversity — but that diversity is organized into a hierarchy we observe everywhere.

Using a human roles example (students, working professionals, college students, software developers, full-stack developers), we see that hierarchies exist naturally. The core idea of **inheritance** is that classes lower in the hierarchy inherit features and attributes from their ancestors. For instance, since all working professionals draw a monthly salary, software developers and full-stack developers inherit that salary attribute from their ancestors.

Concrete Example

Assignments are collections of questions, and questions come in different types (NAT, MCQ). Though a NAT differs from an MCQ, both are questions — forming a hierarchy. Since parents come first, the base `Question` class is defined:

PYTHON

```
class Question:
    def __init__(self, statement, marks):
        self.statement = statement
        self.marks = marks

    def print_question(self):
        print(self.statement)

    def update_marks(self, marks):
        self.marks = marks
```

Only attributes common to all questions are retained: the statement and the marks. Next, child classes for MCQ and NAT are defined using the hierarchy relationship:

PYTHON

```
class NAT(Question):
    def __init__(self, statement, marks, answer):
        super().__init__(statement, marks)
        self.answer = answer

    def update_answer(self, answer):
        self.answer = answer
```

A `NAT` is a specialized `Question` with an extra feature (`answer`) and a new method (`update_answer`), while inheriting everything else. `Question` becomes the "parent-class or base-class" and `NAT` the "child-class or derived-class."

The general syntax:

PYTHON

```
class Derived(Base):
    def __init__(self, ...):
        pass

#### OR ####

class Child(Parent):
    def __init__(self, ...):
        ...
```

For this example specifically:

PYTHON

```
class NAT(Question):
    def __init__(self, ...):
        pass
```

Parent-child relationship

Something interesting happens in the derived class constructor:

PYTHON

```
class NAT(Question):
    def __init__(self, statement, marks, answer):
        super().__init__(statement, marks)
        self.answer = answer

    def update_answer(self, answer):
        self.answer = answer
```

The `super()` function points to the parent class (`Question`), so the parent's constructor is effectively called. Inherited methods like `update_marks()` can be invoked directly:

PYTHON

```
q_nat = NAT('What is 1 + 1?', 1, 2)
q_nat.update_marks(4)
print(q_nat.marks)
```

OUTPUT

```
4
```

Method Overriding

The parent class again for reference:

PYTHON

```
class Question:
    def __init__(self, statement, marks):
        self.statement = statement
        self.marks = marks

    def print_question(self):
        print(self.statement)

    def update_marks(self, marks):
        self.marks = marks
```

Sometimes you want to modify inherited method behavior. For an MCQ, printing only the statement is incomplete — options should also appear. You can **override** the inherited method:

PYTHON

```
class MCQ(Question):
    def __init__(self, statement, marks, ops, c_ops):
        super().__init__(statement, marks)
        self.ops = ops      # List of all options
        self.c_ops = c_ops  # List of correct options

    def print_question(self):
        super().print_question()
        # Assume there are only four options
        op_index = ['(a)', '(b)', '(c)', '(d)']
        for i in range(4):
            print(op_index[i], self.ops[i])
```

Since `Question` already prints the statement, the override "piggy-backs" on that behavior via `super()`, then adds the options. Creating and calling an MCQ object:

PYTHON

```
q_mcq = MCQ('What is the capital of India?',
            2,
            ['Chennai', 'Mumbai', 'Kolkata', 'New Delhi'],
            ['New Delhi'])
q_mcq.print_question()
```

OUTPUT

```
What is the capital of India?
(a) Chennai
(b) Mumbai
(c) Kolkata
(d) New Delhi
```

Lesson 8.4 — Vector: A Complete OOP Example

This lesson presents a final example of classes in action, demonstrating several important OOP concepts through the example of a vector.

Vector: Mathematical Preliminaries

Every point $P(x, y)$ in 2D space can be associated with a **vector** — geometrically, a directed arrow from the origin to point P . One tip is always at the origin, and the other tip (the *head*) is at point P . The lesson uses P_1 and P_2 as example vectors.

Key vector operations covered:

Magnitude. The magnitude of a vector $P(x, y)$ is the length of segment OP :

$$|OP| = \sqrt{x^2 + y^2}$$

Scale. Scaling by value s changes the vector's length but not its direction:

$$s \cdot (x, y) \rightarrow (sx, sy)$$

Add. Two vectors combine component-wise:

$$(x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2)$$

Reflect. Reflection about an axis is a special case of rotation about the origin. Reflecting about the X-axis:

$$(x, y) \rightarrow (x, -y)$$

Vector: Specification

The **specification** is the bridge to move from a mathematical vector to a programmatic one — a written description of the attributes and methods the class needs, derived from the mathematical vector.

Attributes:

- `x` : the x-coordinate
- `y` : the y-coordinate

These two attributes fully define any 2D vector.

Methods:

- `__init__()` : constructor; populates attributes from arguments
- `__str__()` : returns coordinates as `(x, y)` for `str()` / `print()`
- `magnitude()` : returns the magnitude
- `scale()` : scales the current vector by `s`
- `rotate_xaxis()` : reflects the current vector about the X-axis
- `rotate_yaxis()` : reflects the current vector about the Y-axis
- `add()` : accepts a vector argument; returns the sum with the current vector

Vector: Definition

PYTHON

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def print(self):
        return f'({self.x},{self.y})'
    def magnitude(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5
    def scale(self, s):
        self.x, self.y = self.x * s, self.y * s
    def rotate_xaxis(self):
        self.y = -self.y
    def rotate_yaxis(self):
        self.x = -self.x
    def add(self, P):
        result = Vector(0, 0)
        result.x, result.y = self.x + P.x, self.y + P.y
        return result
```

All methods except `add()` and `__str__()` return no value — they transform the vector itself. The `add()` method is interesting: it accepts a vector `P`, creates a new zero-vector inside, stores the sum in `result`, and returns it.

Collection of Vectors

The point of having a class is to create objects, since the class is merely a template. Example use case:

PYTHON

```
triangle = [Vector(0, 1), Vector(3, 1), Vector(3, 0)]
```

Here `triangle` is a list of `Vector` objects representing a triangle. To compute the side lengths:

PYTHON

```
def dist(P1, P2):
    return ((P1.x - P2.x) ** 2 + (P1.y - P2.y) ** 2) ** 0.5

def side_lengths(triangle):
    la = dist(triangle[0], triangle[1])
    lb = dist(triangle[1], triangle[2])
    lc = dist(triangle[2], triangle[0])
    return la, lb, lc
```

Similarly, a square could be defined as a list of four vectors. This closes the discussion on object-oriented programming in Python; these concepts will be covered in greater detail when studying Java.

NOTE

The class specification references `__str__()`, but the code above defines a `print()` method instead — reproduced as it appears in the source.